Vidyalankar Institute of Technology

DevOps Project Report

Broker Lead Management System

Consolidated Report for Weeks 1 to 15

Submitted by

Eshanika Amballa

Roll number 2310B0056

Branch CMPN B

Under the guidance of

Dr. Snehal Andhare

October 2026

Evidence completion copy

Screenshot spaces and capture instructions are included throughout the report.

Project overview

Abstract

The Broker Lead Management System is a web application for recording enquiries, updating lead details, searching a pipeline and reviewing lead counts by status. It was selected as the application for a fifteen-week DevOps project because its everyday workflows are small enough to implement while still exercising a database, authentication, browser testing and a repeatable delivery process.

The application uses React and Vite for the frontend, Spring Boot for its REST API and PostgreSQL for persistent application data. Its delivery configuration includes Maven, Git and GitHub, Jenkins, Selenium WebDriver, Docker Compose and Ansible. The report follows the fifteen tasks in the supplied assignment brief and incorporates the problem definition, planning and architecture work already prepared for Weeks 1, 2 and 3.

Current local validation recorded 31 backend tests, 15 Selenium tests and five frontend API tests passing with no skipped tests. The backend JAR and frontend production build also completed successfully. Browser testing used an isolated H2 database, so those results should not be described as evidence of a live PostgreSQL, Jenkins or provisioned-server deployment.

Each weekly chapter explains the purpose of the task, the relevant design, a practical procedure, verification criteria and the evidence required. Sixty numbered screenshot spaces are provided, with instructions beside each one. The final appendices collect the runbooks and identify remaining requirements so the report can be completed without inventing execution logs or approval records.

Keywords

Lead management, DevOps, Git, Jenkins, Selenium, Spring Boot, React, Docker, PostgreSQL and Ansible.

Student review

Declaration and completion details

I submit this report as the consolidated account of the Broker Lead Management System DevOps project. The report identifies the source documents and repository files used to prepare it. Screenshots, execution records, signatures and approval dates will be added only from the actual project activities they describe.

This copy includes blank screenshot spaces and clearly identified verification work. A procedure describes how to perform a task; it does not certify that the task has already been executed. Before signing this declaration, I will check the final contents, replace the required evidence spaces and confirm that the statements match the submitted revision.

Item	Details
Student	Eshanika Amballa
Roll number	2310B0056
Branch	CMPN B
Institution	Vidyalankar Institute of Technology
Professor	Dr. Snehal Andhare
Submission date	_____
Final source revision	_____
Final release tag	_____

Student signature _____

Professor review or approval _____

Acknowledgement I acknowledge the guidance of Dr. Snehal Andhare and the project evaluation process at Vidyalankar Institute of Technology. The technical decisions and remaining work are described in the following chapters.

Navigation

Contents

Problem Definition and Scope 9
Agile Planning and DevOps Workflow 15
Requirements Architecture and Technology Setup 21
Git and GitHub Repository Initialization 27
Feature Development with Branching 33
MVP Completion and Git Collaboration 39
Jenkins Installation and Continuous Integration Job 45
Pipeline as Code and Server Deployment 51
Selenium Test Design and Local Execution 57
Continuous Testing in Jenkins 63
Docker Image and Container Lifecycle 69
Jenkins Docker Continuous Deployment 75
Configuration Management Script 81
Automated Provisioning and Reliability Validation 87
Final Release Documentation and Viva 93
Local development runbook 99
Docker demonstration runbook 100
Selenium execution and failure capture 101
Jenkins gate and controlled defect exercise 102
Ansible control machine and first provisioning 103
Idempotency deployment and rollback runbook 104
Authentication and user API reference 105
Lead and health API reference 106
Data model and source reconciliation 107
Open requirements and completion work 108
Screenshot register for Weeks 1 to 8 109
Screenshot register for Weeks 8 to 15 110
Troubleshooting and viva preparation 111
Sources and final submission checklist 112
Business stories and acceptance conditions 113
Stakeholders scope and Definition of Done 114
Nonfunctional requirements and measurement plan 115
Use case diagram and role boundaries 116

Using this report Each week contains four written pages followed by two evidence pages. Appendix K and Appendix L provide a single screenshot register. Runbooks in the other appendices explain the commands and missing demonstrations in more detail.

Assignment coverage

The fifteen weekly tasks

Week	Task	Main output
1	Problem Definition and Scope	Problem and bounded scope
2	Agile Planning and DevOps Workflow	Stories and delivery plan
3	Requirements Architecture and Technology Setup	SRS and architecture
4	Git and GitHub Repository Initialization	Repository foundation
5	Feature Development with Branching	First reviewed feature
6	MVP Completion and Git Collaboration	MVP and collaboration
7	Jenkins Installation and Continuous Integration Job	Triggered CI build
8	Pipeline as Code and Server Deployment	Pipeline and deployment
9	Selenium Test Design and Local Execution	Local browser tests
10	Continuous Testing in Jenkins	Required browser quality gate
11	Docker Image and Container Lifecycle	Images and container lifecycle
12	Jenkins Docker Continuous Deployment	Versioned continuous deployment
13	Configuration Management Script	Host configuration automation
14	Automated Provisioning and Reliability Validation	Idempotency and recovery
15	Final Release Documentation and Viva	Final release and viva

The assignment brief lists fifteen task blocks. This report treats them as Weeks 1 through 15 in the same order. The brief is a source of assessment requirements, not evidence that any activity has been completed.

Evidence status

Verified results and remaining proof

The following results come from the current local project checks on 28 October 2026. They are recorded separately from the screenshots that still need to be captured for the academic submission.

Check	Recorded result	Boundary
Backend tests	31 passed and zero skipped	JUnit integration, security and service tests
Browser tests	15 passed and zero skipped	Isolated H2-backed test application
Frontend API tests	5 passed and zero skipped	Node test runner for the shared API helper
Backend package	Maven package succeeded	Executable JAR generated locally
Frontend production build	Vite build succeeded	Static assets generated locally
Compose and playbooks	Configuration and YAML validated	No live Docker or Ansible deployment verified

Evidence still needed

Capture actual GitHub reviews, branch protection, Jenkins triggers and runs, registry tags, healthy containers, provisioning recaps, an idempotent rerun and a compatible rollback. The current Jenkins stage order does not yet prove that browser failures block release deployment, and automatic screenshots on Selenium failures are not implemented.

Recent code fixes were present in the working tree during validation. Record the final committed revision before submission. Do not label a historical release tag as the tested current revision without checking that it contains the same changes.

Architecture

Current application architecture

The browser communicates with a versioned REST API. The backend is responsible for password verification, session identity, role checks, validation and database access. Application data is stored in PostgreSQL; H2 is used only for isolated tests.

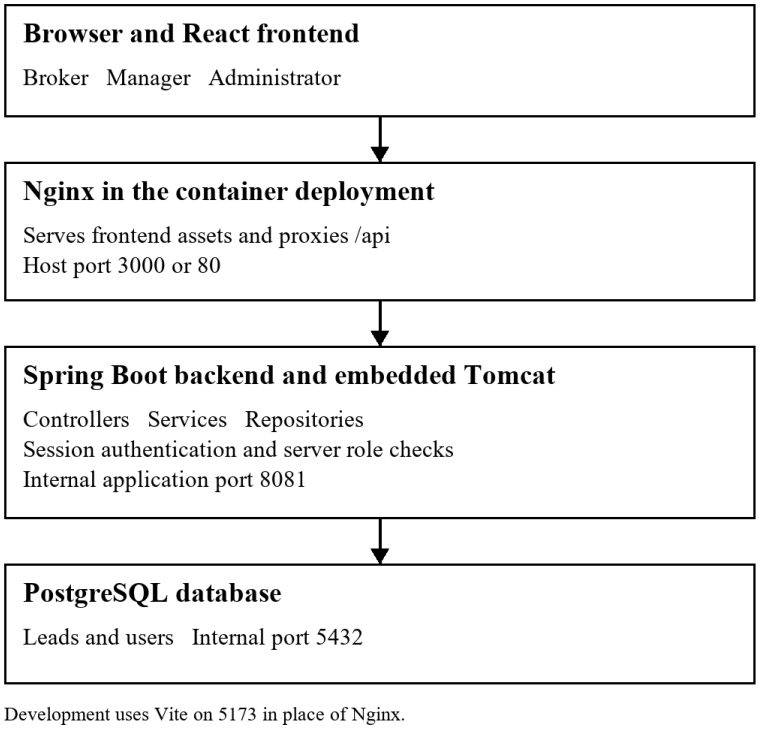


Figure A1 Architecture reconstructed from the current configuration. HTTPS termination, external notifications and a separate Tomcat WAR deployment are not claimed.

In local development, Vite uses port 5173 and proxies to backend port 8081. In Compose, Nginx is available on host port 3000 or 80. The backend and PostgreSQL host mappings are bound to loopback, while the containers communicate on their internal network.

Requirements

Requirements and implementation traceability

ID	Requirement	Current assessment
FR1	Create a lead	Implemented; name required and supported field validation present.
FR2	View leads assigned to a broker	Listing works; broker ownership scoping remains incomplete.
FR3	Edit details and status	Implemented for the supported operations and permissions.
FR4	Search and filter	Keyword/status/source filtering exists; no date-range filter or server pagination.
FR5	Role-based strict workflow	Closing is role-restricted; strict next-state transitions remain incomplete.
FR6	Summary dashboard	Implemented and covered by dashboard tests.
FR7	Role access control	Server checks, account activation and role refresh implemented.
FR8	Secure login	BCrypt, server sessions, CSRF and logout implemented.
FR9	Actor and timestamp audit	Timestamps populated; createdBy/updatedBy not populated by the service.

Original success targets include lead entry under two minutes and dashboard updates within five seconds. No formal timing benchmark or zero-data-loss guarantee is asserted. Passing functional tests supports the exercised paths, not an unlimited reliability claim.

The earlier reports mention MySQL, application.yml, port 8080, planned fields and broader API parameters. The current reference is PostgreSQL, application.properties, port 8081 and the implemented endpoint list in the appendices.

Week 1

Problem Definition and Scope

Purpose of the milestone

A small brokerage can receive enquiries through phone calls, referrals, property portals and walk-ins. When each broker keeps a separate spreadsheet or notebook, the team loses a reliable view of what has already happened. A prospect may be contacted twice, a promised follow-up may be missed, and the manager may have to collect several files just to prepare a pipeline summary.

BLMS addresses this problem by keeping lead details and their current status in one web application. The aim is deliberately limited: capture a lead, find it again, update its details and status, and understand the number of leads in each stage. The project also uses this application to demonstrate a complete DevOps delivery process over fifteen weeks.

Assignment requirement

Study the need for BLMS, identify users and stakeholders, define constraints and measurable success criteria, and freeze a small MVP.

Expected deliverables

- Record the problem, its practical consequences and the people affected by it.
- Separate essential lead management functions from features that would make the project too large.
- Define measurable outcomes that can later be checked with application and delivery evidence.
- Treat scope approval as a review decision that needs an actual signature or recorded acceptance.

Connection to the project

This week has no technical prerequisite. Its output becomes the basis for the backlog, requirements and acceptance tests in Weeks 2 and 3.

Week 1

Technical approach and design decisions

The primary users are brokers, managers and an administrator. Brokers need a straightforward way to record enquiries and maintain contact details. Managers need a consistent summary and authority over closing a lead. The administrator maintains accounts and access. The developer and evaluator are delivery stakeholders rather than daily sales users.

The original Week 1 document sets a fifteen-week schedule, open-source tooling, synthetic data and a Tomcat or Nginx deployment target. These constraints remain useful. The scope includes creation, viewing, editing, searching, status handling and a summary dashboard. Notifications, external lead-source integrations, advanced analytics, a native mobile app and multi-branch operation remain outside the MVP.

Decision	Reason or implementation
Central lead record	Reduce conflicting spreadsheets and forgotten enquiries.
Three application roles	Separate normal brokerage work from closing and account administration.
Small web MVP	Keep implementation and delivery work realistic within fifteen weeks.
Synthetic data	Demonstrate the system without using customer information.
Measured success	Check persistence, usability, tests and delivery instead of relying on a visual demo.

Relevant project artefacts

Supplied Week 1 problem definition and scope document

README.md

docs/BACKLOG.md

Use these files to explain the implementation during review. A code view establishes what is defined; the evidence pages separately request the execution record where the brief requires one.

Week 1

Implementation and demonstration procedure

The following sequence is a practical route through this milestone. Check the existing project state first. Historical activities should be supported by genuine history, and any new demonstration should be identified as a new run.

1. Read the original Week 1 report and list the business problems it identifies.

2. Walk through a sample enquiry from first contact to a closed or lost outcome.

3. Identify who creates records, who reviews the pipeline and who manages accounts.

4. Write the in-scope and excluded functions before estimating development effort.

5. Attach a measurement method to each objective and record any requirement that is still incomplete.

6. Review the scope with the professor or evaluator and retain the actual approval record if one is provided.

Commands or files to inspect

```
Get-Content README.md
Get-Content docs/BACKLOG.md
```

Recording the result

Record the source revision, environment and actual outcome alongside the screenshot. If a command fails, retain the error and explain the correction. Replace example host names, tags and paths with the values for the environment being demonstrated. Keep passwords and tokens outside the captured view.

Run context Date _____ Machine or target _____ Revision _____

Week 1

Verification results and discussion

Check	How to verify	Result or evidence state
Lead entry time	Time a broker entering a realistic synthetic enquiry.	Target under two minutes; timing evidence still needed.
Core data persistence	Create and edit a unique lead, then reload the page.	Browser creation and reload test passed locally.
Dashboard freshness	Change a status and time the updated summary.	Accuracy can be demonstrated; five-second target is not benchmarked.
Audit completeness	Inspect createdBy, updatedBy and timestamps.	Timestamps exist; actor identifiers are not populated by the service.
Scope agreement	Match the final feature list to recorded review feedback.	Written scope exists; approval screenshot or signature is still needed.

Current outcome

The supplied Week 1 report establishes the business problem, users, objectives and boundaries. It does not by itself prove that every proposed success criterion has been achieved. In particular, per-broker ownership and a complete actor audit trail remain gaps in the current implementation.

Discussion

A usable scope gives later weeks a concrete purpose. A dashboard, test suite or container is easier to justify when it supports a named user need. The main lesson is to separate a requirement from proof that the requirement is satisfied.

Completion decision

The milestone can be presented as complete only when its required outputs and the relevant evidence agree. Review the four screenshot requests on the next two pages, fill in the observed identifiers and keep any unresolved requirement visible in the final checklist.

Reviewer comments _____

Week 1

Screenshot evidence A

Insert genuine screenshots in the spaces below. Keep the interface or command output readable and retain the identifiers that connect the image to the activity.

W01-S01 Problem statement and scope

How to capture Open the supplied Week 1 document in Word. Show the problem statement and the approved MVP scope sections at a readable zoom.

What must be visible The title, student details and actual in-scope functions should be visible. Do not add an approval signature that has not been given.

[Insert screenshot W01-S01 here]

Figure W01-S01 Problem statement and scope. Capture date _____ Build or revision _____

W01-S02 Stakeholders and objectives

How to capture Open the stakeholder table and objectives in the Week 1 document. Use two adjacent pages or a single readable document view.

What must be visible Show brokers, managers, the administrator and the measurable objectives.

[Insert screenshot W01-S02 here]

Figure W01-S02 Stakeholders and objectives. Capture date _____ Build or revision _____

Week 1

Screenshot evidence B

Insert genuine screenshots in the spaces below. Keep the interface or command output readable and retain the identifiers that connect the image to the activity.

W01-S03 Success criteria

How to capture Show the success-criteria section. Add a separate note or table recording which criteria have supporting evidence and which are pending.

What must be visible Capture the measurement method, not a fabricated timing result.

[Insert screenshot W01-S03 here]

Figure W01-S03 Success criteria. Capture date _____ Build or revision _____

W01-S04 Scope review record

How to capture If your professor reviewed the scope, open the genuine signed page, feedback message or review note. Otherwise leave this slot pending.

What must be visible Hide unrelated personal information. A prepared report is not the same as approval.

[Insert screenshot W01-S04 here]

Figure W01-S04 Scope review record. Capture date _____ Build or revision _____

Week 2

Agile Planning and DevOps Workflow

Purpose of the milestone

The scope becomes manageable when it is divided into user stories with observable outcomes. A story such as creating a lead needs more than a form: the saved record must appear in the list, invalid input must be rejected, and the data must remain after a reload. These conditions connect planning directly to later tests.

The supplied Week 2 document contains eight business stories, a Definition of Done and a lifecycle description. It refers to a separate BLMS_Sprint_Plan.xlsx workbook for the backlog, board and weekly plan. That workbook was not among the supplied files, so its contents and current status cannot be assumed. The consolidated report provides a planning baseline, while a screenshot of the actual workbook or GitHub board is still required.

Assignment requirement

Create user stories and acceptance criteria, a backlog, a fifteen-week Kanban or Scrum plan, a Definition of Done and a DevOps lifecycle diagram.

Expected deliverables

- Write stories from the broker, manager and administrator perspectives.
- Prioritise essential functions before optional reporting or integration work.
- Track progress using Backlog, To Do, In Progress, In Review and Done states.
- Connect planning to code review, automated tests, release and operational feedback.

Connection to the project

Uses the Week 1 scope. The resulting stories and Definition of Done guide feature development and the test plan.

Week 2

Technical approach and design decisions

The original story identifiers and the repository backlog identifiers are not fully interchangeable. For example, the original Week 2 US-05 concerns manager workflow control, while the repository backlog uses a different ordering. This report uses functional requirement identifiers in its traceability matrix so the meaning remains clear even when a story number changes.

A task is Done only when its acceptance criteria have been checked, relevant tests pass, review is complete and documentation reflects the result. A green local build is useful evidence, but it cannot stand in for a Jenkins run or an approved pull request. The task board should therefore distinguish implemented work from evidence that still needs to be collected.

Decision	Reason or implementation
Must have	Lead CRUD, search, login, role checks and summary counts.
Should have	Clear errors, automated regression coverage and repeatable builds.
Could have	CSV export and additional presentation improvements.
Outside MVP	Notifications, external lead ingestion and multi-tenancy.
Definition of Done	Acceptance checked, reviewed, tested, documented and merged appropriately.

Relevant project artefacts

Supplied Week 2 PDF

docs/BACKLOG.md

CONTRIBUTING.md

BLMS_Sprint_Plan.xlsx if available

Use these files to explain the implementation during review. A code view establishes what is defined; the evidence pages separately request the execution record where the brief requires one.

Week 2

Implementation and demonstration procedure

The following sequence is a practical route through this milestone. Check the existing project state first. Historical activities should be supported by genuine history, and any new demonstration should be identified as a new run.

1. Convert each scope item into a story with a user, capability and reason.

2. Write at least one positive and one negative acceptance condition for core data-entry work.

3. Assign a priority, estimated effort and target week to each backlog item.

4. Create or update a GitHub Projects board using the five planning states.

5. Record dependencies between application, testing, container and provisioning tasks.

6. Review the board at the end of each milestone and move a task to Done only when its evidence is available.

Commands or files to inspect

```
Get-Content docs/BACKLOG.md
Get-Content CONTRIBUTING.md
git log --all --oneline --decorate -15
```

Recording the result

Record the source revision, environment and actual outcome alongside the screenshot. If a command fails, retain the error and explain the correction. Replace example host names, tags and paths with the values for the environment being demonstrated. Keep passwords and tokens outside the captured view.

Run context Date _____ Machine or target _____ Revision _____

Week 2

Verification results and discussion

Check	How to verify	Result or evidence state
Story quality	Read each story and its Given When Then conditions.	Eight original business stories are available in the supplied PDF.
Backlog coverage	Map all fifteen weekly tasks to board items.	Repository backlog exists; board or workbook proof is pending.
Priority clarity	Separate required work from optional enhancements.	Use the scope and priority table as the baseline.
Definition of Done	Check review, tests, documentation and merge conditions.	Written definition exists; completion is task-specific.
Lifecycle consistency	Compare the diagram to the actual Jenkins stage order.	Current browser tests occur after deployment; a release gate adjustment remains needed.

Current outcome

The planning documents explain the intended workflow and provide acceptance criteria. An actual board, its history and the companion workbook need to be supplied or recreated honestly. The report does not infer sprint completion dates or team meetings from a static backlog.

Discussion

A board is useful when it reveals unfinished work. Marking a task Done because code exists hides missing review, test or deployment evidence. Acceptance criteria make those differences visible before the final submission.

Completion decision

The milestone can be presented as complete only when its required outputs and the relevant evidence agree. Review the four screenshot requests on the next two pages, fill in the observed identifiers and keep any unresolved requirement visible in the final checklist.

Reviewer comments _____

Week 2

Screenshot evidence A

Insert genuine screenshots in the spaces below. Keep the interface or command output readable and retain the identifiers that connect the image to the activity.

W02-S01 Product backlog

How to capture Open your GitHub Projects backlog or the original planning workbook. Show story title, priority, status and target week.

What must be visible Include the project name and a readable selection of real backlog items.

[Insert screenshot W02-S01 here]

Figure W02-S01 Product backlog. Capture date _____ Build or revision _____

W02-S02 Kanban task board

How to capture Open the board view with Backlog, To Do, In Progress, In Review and Done columns. Update cards to their actual state.

What must be visible Do not place pending evidence or untested deployment work in Done.

[Insert screenshot W02-S02 here]

Figure W02-S02 Kanban task board. Capture date _____ Build or revision _____

Week 2

Screenshot evidence B

Insert genuine screenshots in the spaces below. Keep the interface or command output readable and retain the identifiers that connect the image to the activity.

W02-S03 Fifteen week plan

How to capture Show the weekly plan in the workbook or GitHub milestone view. Link the fifteen tasks to their deliverables.

What must be visible The plan should cover Weeks 1 through 15 without invented completion dates.

[Insert screenshot W02-S03 here]

Figure W02-S03 Fifteen week plan. Capture date _____ Build or revision _____

W02-S04 Definition of Done and workflow

How to capture Show the Definition of Done beside the lifecycle diagram from the Week 2 report or your updated planning document.

What must be visible Make the review and test conditions readable. Label a proposed workflow as proposed.

[Insert screenshot W02-S04 here]

Figure W02-S04 Definition of Done and workflow. Capture date _____ Build or revision _____

Week 3

Requirements Architecture and Technology Setup

Purpose of the milestone

This milestone translates the business scope into interfaces and responsibilities. The browser should handle interaction, the backend should validate requests and enforce permissions, and the database should hold persistent records. This separation allows the frontend and backend to be built independently while still supporting one complete user journey.

The current project uses React 18 and Vite, Java with Spring Boot 3.2.5, Spring Data JPA, Spring Security and PostgreSQL. Maven produces an executable JAR with embedded Tomcat. Nginx serves the frontend production build and forwards API requests. This satisfies the selected deployment approach without requiring a separate WAR deployment.

Assignment requirement

Produce an SRS summary, use cases, architecture, a data model and API list, and establish a working local development setup.

Expected deliverables

- Translate the scope into functional and non-functional requirements.
- Define the three application actors and their permitted operations.
- Specify the request path, persistent entities and versioned API.
- Check that the local frontend, backend and database can communicate.

Connection to the project

Uses the Week 1 scope and Week 2 acceptance criteria. Establishes the technical baseline for all later milestones.

Week 3

Technical approach and design decisions

In development, Vite serves the frontend on port 5173 and proxies /api to backend port 8081. In the container stack, Nginx is reached on host port 3000 or 80, the backend listens internally on 8081 and PostgreSQL uses 5432. Only the backend talks directly to the database. The frontend uses a server session cookie and obtains a CSRF token before state-changing requests.

The original Week 3 PDF contains some early setup inconsistencies. Its references to MySQL, application.yml and backend port 8080 do not describe the current checkout. The actual configuration file is application.properties and the actual database is PostgreSQL. Ownership fields exist on Lead, but foreign-key enforcement, broker scoping and actor audit population are not currently established by the service.

Decision	Reason or implementation
Frontend	React components call the REST API through a shared request helper.
Application layer	Controllers, services and repositories separate HTTP, business logic and persistence.
Authentication	BCrypt password verification followed by an HttpOnly server session.
Deployment	Executable Spring Boot JAR with embedded Tomcat and an Nginx frontend.
Persistence	PostgreSQL for the application; H2 for isolated automated tests.

Relevant project artefacts

Supplied Week 3 PDF

backend/pom.xml

backend/src/main/resources/application.properties

frontend/vite.config.js

frontend/nginx.conf

Use these files to explain the implementation during review. A code view establishes what is defined; the evidence pages separately request the execution record where the brief requires one.

Week 3

Implementation and demonstration procedure

The following sequence is a practical route through this milestone. Check the existing project state first. Historical activities should be supported by genuine history, and any new demonstration should be identified as a new run.

1. Review the SRS and match every requirement to an actor and a measurable outcome.
2. Draw the request path from browser to Nginx, backend and PostgreSQL.
3. Inspect Lead and User and document the actual field names and constraints.
4. List the implemented API endpoints and distinguish them from planned query parameters.
5. Configure database credentials without placing passwords in screenshots or committed files.
6. Start backend and frontend, check health and create a synthetic lead that survives a page reload.

Commands or files to inspect

```
java -version
mvn -version
node --version
npm --version
curl.exe http://localhost:8081/api/v1/health
npm run dev
```

Recording the result

Record the source revision, environment and actual outcome alongside the screenshot. If a command fails, retain the error and explain the correction. Replace example host names, tags and paths with the values for the environment being demonstrated. Keep passwords and tokens outside the captured view.

Run context Date _____ Machine or target _____ Revision _____

Week 3

Verification results and discussion

Check	How to verify	Result or evidence state
Versioned API	Inspect controller mappings under /api/v1.	Implemented in the current source.
Local build	Run Maven tests/package and the Vite production build.	Both passed in the local validation session.
Database connection	Call the health endpoint with PostgreSQL running.	Endpoint checks SELECT 1; current PostgreSQL runtime proof is pending.
Role permissions	Try broker, manager and admin actions.	Security integration tests passed locally.
Architecture accuracy	Compare diagrams and setup notes to current configuration.	Use PostgreSQL, port 8081 and executable JAR in the final report.

Current outcome

The source establishes a working layered application and the selected stack. Isolated H2 tests and browser checks passed. A new screenshot of the normal PostgreSQL development setup is still needed, because the isolated test environment is not proof of a live PostgreSQL deployment.

Discussion

An architecture document must describe the implemented system precisely enough for another person to run it. A small mismatch in a port, database driver or configuration filename can make otherwise correct instructions unusable.

Completion decision

The milestone can be presented as complete only when its required outputs and the relevant evidence agree. Review the four screenshot requests on the next two pages, fill in the observed identifiers and keep any unresolved requirement visible in the final checklist.

Reviewer comments _____

Week 3

Screenshot evidence A

Insert genuine screenshots in the spaces below. Keep the interface or command output readable and retain the identifiers that connect the image to the activity.

W03-S01 Tools and versions

How to capture Open PowerShell in the repository and run java -version, mvn -version, node --version and npm --version.

What must be visible Show the actual versions. The source targets Java 17 while this workstation used JDK 21.

[Insert screenshot W03-S01 here]

Figure W03-S01 Tools and versions. Capture date _____ Build or revision _____

W03-S02 Backend and database health

How to capture Start PostgreSQL and the backend with the configured credentials. Run curl.exe http://localhost:8081/api/v1/health.

What must be visible Show the backend startup and the UP response. Keep passwords out of the terminal view.

[Insert screenshot W03-S02 here]

Figure W03-S02 Backend and database health. Capture date _____ Build or revision _____

Week 3

Screenshot evidence B

Insert genuine screenshots in the spaces below. Keep the interface or command output readable and retain the identifiers that connect the image to the activity.

W03-S03 Working local frontend

How to capture Start the frontend with npm run dev. Open http://localhost:5173 and sign in with a real account.

What must be visible Show the application address and a lead list returned by the backend.

[Insert screenshot W03-S03 here]

Figure W03-S03 Working local frontend. Capture date _____ Build or revision _____

W03-S04 Architecture and data model

How to capture Open the architecture page and the Lead and User classes, or show your database table structure.

What must be visible Use current names and ports. Do not describe nullable identifier fields as enforced relationships.

[Insert screenshot W03-S04 here]

Figure W03-S04 Architecture and data model. Capture date _____ Build or revision _____

Week 4

Git and GitHub Repository Initialization

Purpose of the milestone

Version control makes the application and its delivery configuration reviewable together. The backend, frontend, Docker definitions, Jenkins pipeline and Ansible playbooks belong to the same repository so a particular revision can explain how a release was built and operated.

The local checkout has a configured origin at github.com/esh22nika/broker_lead_management. It contains README.md, .gitignore, CONTRIBUTING.md and bug and feature issue templates. These files establish conventions, but their presence does not prove that GitHub branch protection is enabled or that a particular review took place. Those details require evidence from GitHub itself.

Assignment requirement

Create the repository, README, ignore rules, folders, issue templates and branch policy, then commit the initial application skeleton.

Expected deliverables

- Create a clear repository structure for application code and operations files.
- Keep generated builds, private configuration and credentials out of version control.
- Write a README that another developer can follow.
- Define meaningful commit messages, issue formats and branch naming rules.

Connection to the project

Uses the architecture and folder choices from Week 3. Provides the shared baseline for feature branches and CI.

Week 4

Technical approach and design decisions

The repository separates backend, frontend, docs and ansible directories, with shared delivery files at the root. The ignore file excludes Maven targets, node_modules, local environment files, private keys and logs. A sample environment file documents the required variables without providing a usable password.

CONTRIBUTING.md describes main, develop, feature, release and hotfix branches and a pull request process. Treat this as a policy document. GitHub settings must separately enforce the protections that are required for assessment. The current working tree also contains recent uncommitted fixes, so a submission must record the actual committed revision and any remaining local changes.

Decision	Reason or implementation
README	Explain purpose, stack, setup and verification.
Ignore rules	Exclude generated files and private local configuration.
Issue templates	Capture reproducible bug reports and acceptance criteria.
Branch policy	Use reviewable feature work and an integration branch.
Commit messages	Use a short change type and a specific description.

Relevant project artefacts

.gitignore

README.md

CONTRIBUTING.md

.github/ISSUE_TEMPLATE/bug_report.md

.github/ISSUE_TEMPLATE/feature_request.md

Use these files to explain the implementation during review. A code view establishes what is defined; the evidence pages separately request the execution record where the brief requires one.

Week 4

Implementation and demonstration procedure

The following sequence is a practical route through this milestone. Check the existing project state first. Historical activities should be supported by genuine history, and any new demonstration should be identified as a new run.

1. Open the repository on GitHub and check that the project description matches the MVP.

2. Inspect the local folder structure and verify that source files are tracked in the intended directories.

3. Check ignore rules using harmless generated paths rather than adding private files to a commit.

4. Review the README setup commands against the current ports and environment variables.

5. Create a genuine issue for unfinished work and complete the issue template.

6. Check branch protection in GitHub settings and record the settings that are actually enabled.

Commands or files to inspect

```
git remote -v
git status --short
git log --all --oneline --decorate -15
git branch -a
git check-ignore backend/target frontend/node_modules .env
```

Recording the result

Record the source revision, environment and actual outcome alongside the screenshot. If a command fails, retain the error and explain the correction. Replace example host names, tags and paths with the values for the environment being demonstrated. Keep passwords and tokens outside the captured view.

Run context Date _____ Machine or target _____ Revision _____

Week 4

Verification results and discussion

Check	How to verify	Result or evidence state
Repository identity	Compare origin with the repository shown in the browser.	Origin is configured locally; remote screenshot is pending.
Useful documentation	Follow the README from a new terminal.	Setup instructions have been corrected for current code.
Private files excluded	Check .env, target and node_modules ignore behaviour.	Ignore patterns exist; capture command output.
Issue process	Open a real issue with clear acceptance conditions.	Templates exist; issue evidence must come from GitHub.
Protection rules	Inspect main and develop in repository settings.	Documentation alone is not proof of enforced protection.

Current outcome

The required repository files are present and the remote is configured. Initial commits and their sequence should be shown using the actual history. The report leaves branch protection, issue creation and reviewer activity as evidence to capture rather than inferring them from a contribution guide.

Discussion

A repository becomes useful to another developer when its structure and instructions are understandable without private explanations. Review rules and ignore patterns are small files, but they have a direct effect on reproducibility and safe collaboration.

Completion decision

The milestone can be presented as complete only when its required outputs and the relevant evidence agree. Review the four screenshot requests on the next two pages, fill in the observed identifiers and keep any unresolved requirement visible in the final checklist.

Reviewer comments _____

Week 4

Screenshot evidence A

Insert genuine screenshots in the spaces below. Keep the interface or command output readable and retain the identifiers that connect the image to the activity.

W04-S01 Repository home

How to capture Open https://github.com/esh22nika/broker_lead_management. Show the repository name, README and top-level folders.

What must be visible Capture the real browser page with the repository address visible.

[Insert screenshot W04-S01 here]

Figure W04-S01 Repository home. Capture date _____ Build or revision _____

W04-S02 Initial commit history

How to capture Run git log --reverse --oneline and show the initial skeleton commits, or open the earliest commits on GitHub.

What must be visible Keep commit hashes and meaningful messages readable.

[Insert screenshot W04-S02 here]

Figure W04-S02 Initial commit history. Capture date _____ Build or revision _____

Week 4

Screenshot evidence B

Insert genuine screenshots in the spaces below. Keep the interface or command output readable and retain the identifiers that connect the image to the activity.

W04-S03 Issues and templates

How to capture Create or open a genuine project issue using the bug or feature template.

What must be visible Show the issue number, title and acceptance conditions. Do not invent historical issues.

[Insert screenshot W04-S03 here]

Figure W04-S03 Issues and templates. Capture date _____ Build or revision _____

W04-S04 Branch policy and protection

How to capture Open CONTRIBUTING.md and GitHub Settings Branches or Rules.

What must be visible Show the actual main/develop protections. If you lack settings access, record that limitation.

[Insert screenshot W04-S04 here]

Figure W04-S04 Branch policy and protection. Capture date _____ Build or revision _____

Week 5

Feature Development with Branching

Purpose of the milestone

Creating a lead is a suitable first vertical feature because it crosses every application layer. The user enters details in React, the backend validates the request, the service creates a NEW lead and the repository stores it. A successful feature therefore demonstrates more than a user interface component.

The current code includes CreateLeadForm, CreateLeadModal, CreateLeadRequest, LeadController and LeadService. The frontend now reports a failed request as an error instead of fabricating a local saved record. This is important for the acceptance criterion: a lead should appear because the database accepted it, not because the browser invented a replacement.

Assignment requirement

Implement the first core workflow on a feature branch, use normal Git commands, raise and review a pull request, and merge into development.

Expected deliverables

- Deliver one complete lead-creation journey before expanding the application.
- Use a feature branch to keep the change reviewable.
- Commit related changes with messages that explain the feature.
- Retain a real pull request, comments and merge record as collaboration evidence.

Connection to the project

Requires the repository and local application baseline. Establishes the branch and review process used for the remaining MVP.

Week 5

Technical approach and design decisions

The creation endpoint accepts a name and optional contact details, source and notes. It assigns NEW status and creation/update timestamps on the server. The controller rejects an empty name and validates supported field lengths and email format. State-changing requests use the session and a CSRF token through the shared frontend helper.

The original acceptance criteria described contact information and source as required. The current backend only requires a nonblank name. This is a requirement difference that should be decided explicitly with the evaluator. A test should reflect the agreed rule instead of silently claiming that all original mandatory fields are enforced.

Decision	Reason or implementation
Vertical feature	Connect the form, API, service and database.
Server-generated fields	Assign status and timestamps on the backend.
Failure handling	Keep the form open and show an error when saving fails.
Branch review	Show a focused change and address actual review feedback.
Persistence proof	Reload after creation and verify the unique lead still exists.

Relevant project artefacts

frontend/src/components/CreateLeadForm.jsx

frontend/src/components/CreateLeadModal.jsx

backend/src/main/java/com/blms/controller/LeadController.java

backend/src/main/java/com/blms/service/LeadService.java

Use these files to explain the implementation during review. A code view establishes what is defined; the evidence pages separately request the execution record where the brief requires one.

Week 5

Implementation and demonstration procedure

The following sequence is a practical route through this milestone. Check the existing project state first. Historical activities should be supported by genuine history, and any new demonstration should be identified as a new run.

1. Inspect the existing history before choosing a feature branch to present as evidence.

2. For new demonstration work, branch from the agreed base and make a small genuine improvement.

3. Review the form request and server validation, then run the relevant tests.

4. Create a synthetic lead with a unique name and verify it remains after reloading.

5. Push the branch and open a pull request into the agreed development branch.

6. Address actual review comments, merge according to policy and retain the PR link.

Commands or files to inspect

```
git status --short
git branch -a
git log --all --oneline -- frontend/src/components/CreateLeadForm.jsx
git diff --stat
mvn test
npm test
```

Recording the result

Record the source revision, environment and actual outcome alongside the screenshot. If a command fails, retain the error and explain the correction. Replace example host names, tags and paths with the values for the environment being demonstrated. Keep passwords and tokens outside the captured view.

Run context Date _____ Machine or target _____ Revision _____

Week 5

Verification results and discussion

Check	How to verify	Result or evidence state
New lead	Submit a unique name through the form.	Creation and reload persistence passed in the browser suite.
Blank name	Submit the quick-entry form without a name.	Browser validation test passed.
Invalid contact email	Send an invalid email through an authenticated request.	Security integration suite checks invalid field rejection.
Request failure	Stop the test backend and try saving synthetic data.	Expected error with no saved record; screenshot still needed.
Review and merge	Show the actual PR, feedback and merge commit.	Requires GitHub evidence, not reconstructed screenshots.

Current outcome

The first core workflow exists and automated creation checks passed. A genuine feature branch and pull request should be selected from history or created for new work. The report does not claim that the recent local fixes have already been committed or reviewed.

Discussion

A feature is complete when its data and failure paths work across the stack. Branches and pull requests also need to tell a coherent story: what changed, how it was checked and why it was safe to merge.

Completion decision

The milestone can be presented as complete only when its required outputs and the relevant evidence agree. Review the four screenshot requests on the next two pages, fill in the observed identifiers and keep any unresolved requirement visible in the final checklist.

Reviewer comments _____

Week 5

Screenshot evidence A

Insert genuine screenshots in the spaces below. Keep the interface or command output readable and retain the identifiers that connect the image to the activity.

W05-S01 Feature branch and commits

How to capture Open a genuine feature branch or run git log on the branch that introduced lead creation.

What must be visible Show its relationship to develop or the documented base. Do not rename current work to imply old history.

[Insert screenshot W05-S01 here]

Figure W05-S01 Feature branch and commits. Capture date _____ Build or revision _____

W05-S02 Working create workflow

How to capture Sign in and create a lead named with a unique timestamp, then reload the list.

What must be visible Show the same unique record after reload, including NEW status.

[Insert screenshot W05-S02 here]

Figure W05-S02 Working create workflow. Capture date _____ Build or revision _____

Week 5

Screenshot evidence B

Insert genuine screenshots in the spaces below. Keep the interface or command output readable and retain the identifiers that connect the image to the activity.

W05-S03 Pull request review

How to capture Open the actual PR Files changed and conversation views.

What must be visible Capture the feature diff, real comments and any response or correction.

[Insert screenshot W05-S03 here]

Figure W05-S03 Pull request review. Capture date _____ Build or revision _____

W05-S04 Merge evidence

How to capture Open the merged PR or the corresponding merge commit in GitHub.

What must be visible Show the target branch and commit hash. If no PR exists, leave this evidence pending.

[Insert screenshot W05-S04 here]

Figure W05-S04 Merge evidence. Capture date _____ Build or revision _____

Week 6

MVP Completion and Git Collaboration

Purpose of the milestone

After lead creation, the MVP needs to support the repeated work of managing enquiries. The application includes a list, keyword search, status/source filtering, editing, deletion for permitted roles, a dashboard and user administration. Table and Kanban views present the same lead data in different ways.

The automated checks cover the implemented behaviours, but the original SRS sets a wider boundary in a few areas. Leads are not currently scoped to an assigned broker, audit actor fields are not filled and status permissions do not form a strict transition graph. These are recorded as gaps rather than hidden under a general claim that the MVP is fully complete.

Assignment requirement

Complete the core functions, demonstrate another feature branch, resolve a real merge conflict, tag a baseline and update the backlog.

Expected deliverables

- Connect the remaining lead functions to the persistent backend.
- Check role restrictions through the API as well as through visible controls.
- Demonstrate how conflicting edits are identified and resolved.
- Create a release baseline only after reviewing the intended changes and test results.

Connection to the project

Builds on the first feature and Git workflow. Produces the baseline that Jenkins should build in Week 7.

Week 6

Technical approach and design decisions

Brokers can update ordinary lead statuses, while CONVERTED and LOST require a manager or administrator. Only managers and administrators can delete leads, and user management is administrator-only. The backend rechecks account activation and role on later requests, so changing a user account affects an existing session.

The lead list filters its full loaded dataset without overwriting that dataset with a search result. An empty response replaces old data, and the dashboard status cards pass the selected filter into the pipeline view. A merge-conflict demonstration should use a disposable practice branch or worktree so it cannot damage the active project.

Decision	Reason or implementation
Shared lead dataset	Keep search and display filtering separate from persisted records.
Role enforcement	Reject protected actions on the server.
Conflict resolution	Review both intended edits and test the combined result.
Release baseline	Record the exact tested revision and tag.
Open requirements	Keep ownership, actor auditing and strict transitions visible in the backlog.

Relevant project artefacts

frontend/src/App.jsx

frontend/src/components/LeadsList.jsx

frontend/src/components/Dashboard.jsx

backend/src/main/java/com/blms/config/SecurityConfig.java

docs/BACKLOG.md

Use these files to explain the implementation during review. A code view establishes what is defined; the evidence pages separately request the execution record where the brief requires one.

Week 6

Implementation and demonstration procedure

The following sequence is a practical route through this milestone. Check the existing project state first. Historical activities should be supported by genuine history, and any new demonstration should be identified as a new run.

1. Demonstrate view, edit, search and dashboard behaviour using synthetic records.

2. Repeat protected actions as a broker and as a manager to compare outcomes.

3. Identify a genuine second feature branch and its history.

4. Use a safe practice branch to create conflicting edits to a disposable text file.

5. Resolve the conflict after explaining which parts of each change should remain, then rerun checks.

6. Review the current diff, commit the approved baseline and create an appropriate release tag only when ready.

Commands or files to inspect

```
git branch -a
git log --all --graph --oneline --decorate -25
git status --short
git diff --check
git tag --list
mvn test
```

Recording the result

Record the source revision, environment and actual outcome alongside the screenshot. If a command fails, retain the error and explain the correction. Replace example host names, tags and paths with the values for the environment being demonstrated. Keep passwords and tokens outside the captured view.

Run context Date _____ Machine or target _____ Revision _____

Week 6

Verification results and discussion

Check	How to verify	Result or evidence state
Search and clearing	Search for a unique record, then clear the query.	Both browser search tests passed locally.
Editing	Edit details and reload the list.	API integration tests cover updates; capture manual proof.
Protected actions	Try closing/deleting a lead as a broker.	Backend denial tests passed.
Conflict resolution	Show conflict markers, the resolved file and the resolution commit.	A genuine demonstration is still needed.
Tagged baseline	Show the tag resolving to the tested commit.	Inspect actual tags; do not infer a current release from the changelog.

Current outcome

The application has a functional core and passing local regression tests. The remaining requirement gaps are stated explicitly in the compliance matrix. Historical collaboration and tagging need actual repository evidence, particularly because the current working tree includes uncommitted changes.

Discussion

MVP completion is more useful when described at the level of individual requirements. A polished screen cannot establish access control, and a tag cannot establish that the tagged code is the code currently being demonstrated.

Completion decision

The milestone can be presented as complete only when its required outputs and the relevant evidence agree. Review the four screenshot requests on the next two pages, fill in the observed identifiers and keep any unresolved requirement visible in the final checklist.

Reviewer comments _____

Week 6

Screenshot evidence A

Insert genuine screenshots in the spaces below. Keep the interface or command output readable and retain the identifiers that connect the image to the activity.

W06-S01 Complete lead workflow

How to capture Show the lead list with a unique edited record and the dashboard count after its status changes.

What must be visible Keep the record name, status and relevant summary visible across the captured views.

[Insert screenshot W06-S01 here]

Figure W06-S01 Complete lead workflow. Capture date _____ Build or revision _____

W06-S02 Second feature branch

How to capture Open the branch or PR for a second real feature such as search or dashboard.

What must be visible Show the actual commits and target branch.

[Insert screenshot W06-S02 here]

Figure W06-S02 Second feature branch. Capture date _____ Build or revision _____

Week 6

Screenshot evidence B

Insert genuine screenshots in the spaces below. Keep the interface or command output readable and retain the identifiers that connect the image to the activity.

W06-S03 Conflict and resolution

How to capture In a disposable practice branch, capture the conflict markers first, then the resolved content and commit.

What must be visible Use a genuine conflict. Never edit a screenshot or fabricate a merge log.

[Insert screenshot W06-S03 here]

Figure W06-S03 Conflict and resolution. Capture date _____ Build or revision _____

W06-S04 Release tag and backlog

How to capture Run git tag --list and inspect the selected tag with git show TAG_NAME. Show the updated task board separately if necessary.

What must be visible Replace TAG_NAME with a real tag. The tag must identify the code you tested.

[Insert screenshot W06-S04 here]

Figure W06-S04 Release tag and backlog. Capture date _____ Build or revision _____

Week 7

Jenkins Installation and Continuous Integration Job

Purpose of the milestone

Continuous integration provides a repeatable way to build the repository and report whether its checks pass. A developer should be able to push a change and identify the corresponding Jenkins build, test result and packaged output without repeating a private sequence of terminal commands.

The repository contains a Jenkinsfile, but this assessment did not inspect a running Jenkins controller. Local Maven and npm results establish that the application can be built on this workstation. They do not establish that a Jenkins job is configured, triggered or successful. This week therefore contains a setup procedure and explicit spaces for the real Jenkins evidence.

Assignment requirement

Install and configure Jenkins, connect GitHub, create a triggered build job and archive the produced artefact.

Expected deliverables

- Configure a Jenkins agent with the tools required by the project.
- Connect the job to the real GitHub repository and an appropriate branch.
- Run tests and package the backend as part of an automated build.
- Retain the build trigger and archived JAR as reviewable evidence.

Connection to the project

Requires a buildable repository baseline. Provides the Jenkins installation reused for pipeline and continuous testing work.

Week 7

Technical approach and design decisions

The current pipeline expects a Windows agent labelled windows-docker, Maven configured as Maven-3.9.16 and JDK configured as JDK-21. The source still targets Java 17. Node.js, npm, Chrome and Docker Compose must be available to the agent account because later stages use them.

A webhook or scheduled SCM poll should be tied to a specific repository and branch. The build record should show the triggering cause and checked-out commit. Tests must run under the same workspace that produces the artefact, and an archived JAR should be identifiable by the successful build that created it.

Decision	Reason or implementation
Agent identity	Use a documented Windows agent with the expected label.
Tool names	Match Jenkins Global Tool Configuration to the Jenkinsfile.
Build trigger	Use a real webhook or configured SCM polling interval.
Test reporting	Publish Surefire XML rather than only a success message.
Artefact archive	Keep the packaged JAR with the Jenkins build record.

Relevant project artefacts

Jenkinsfile

backend/pom.xml

backend/target/blms-backend.jar

backend/target/surefire-reports

Use these files to explain the implementation during review. A code view establishes what is defined; the evidence pages separately request the execution record where the brief requires one.

Week 7

Implementation and demonstration procedure

The following sequence is a practical route through this milestone. Check the existing project state first. Historical activities should be supported by genuine history, and any new demonstration should be identified as a new run.

1. Install or open Jenkins on a controlled development machine and complete its normal setup.
2. Configure the named Maven and JDK tools and verify the agent label.
3. Create a job using the Git repository and record the selected branch.
4. Configure a real webhook or an SCM polling schedule.
5. Run the job and inspect compilation, test output and packaging.
6. Open the archived artefact and record the build number, commit and trigger cause.

Commands or files to inspect

```
java -version
mvn -version
node --version
docker version
mvn clean verify -B
```

Recording the result

Record the source revision, environment and actual outcome alongside the screenshot. If a command fails, retain the error and explain the correction. Replace example host names, tags and paths with the values for the environment being demonstrated. Keep passwords and tokens outside the captured view.

Run context Date _____ Machine or target _____ Revision _____

Week 7

Verification results and discussion

Check	How to verify	Result or evidence state
Agent readiness	Run tool-version commands under the Jenkins agent account.	Requires Jenkins agent evidence.
SCM connection	Inspect the checkout log and revision.	Repository URL is known; job execution is not verified here.
Trigger	Push a harmless approved change or wait for the configured poll.	Capture the actual build cause.
Tests	Open the Jenkins JUnit result view.	Expected 31 non-browser backend tests in this revision.
Archived output	Open the artefact list for the same build.	Local JAR packaging passed; Jenkins archive proof is pending.

Current outcome

CI configuration is available in source, and the local build can produce the expected JAR. A successful Jenkins run, automatic trigger and archived artefact must be captured from the actual controller before this milestone is signed off.

Discussion

The agent environment is part of the build. A job can fail even when application code is correct if a tool name, executable path or service-account permission differs from the developer workstation.

Completion decision

The milestone can be presented as complete only when its required outputs and the relevant evidence agree. Review the four screenshot requests on the next two pages, fill in the observed identifiers and keep any unresolved requirement visible in the final checklist.

Reviewer comments _____

Week 7

Screenshot evidence A

Insert genuine screenshots in the spaces below. Keep the interface or command output readable and retain the identifiers that connect the image to the activity.

W07-S01 Jenkins installation and tools

How to capture Open Jenkins dashboard and Manage Jenkins Tools. Show the Maven and JDK names used by the pipeline.

What must be visible Do not show initial admin passwords, credential values or API tokens.

[Insert screenshot W07-S01 here]

Figure W07-S01 Jenkins installation and tools. Capture date _____ Build or revision _____

W07-S02 Job and repository settings

How to capture Open the job configuration showing the SCM repository and the chosen branch.

What must be visible Capture the repository URL and trigger configuration at readable zoom.

[Insert screenshot W07-S02 here]

Figure W07-S02 Job and repository settings. Capture date _____ Build or revision _____

Week 7

Screenshot evidence B

Insert genuine screenshots in the spaces below. Keep the interface or command output readable and retain the identifiers that connect the image to the activity.

W07-S03 Automatic build cause

How to capture Trigger the job through the configured mechanism and open the build console or build summary.

What must be visible Show the cause and checked-out commit, not only a manually started green build.

[Insert screenshot W07-S03 here]

Figure W07-S03 Automatic build cause. Capture date _____ Build or revision _____

W07-S04 Successful build and archive

How to capture Open the successful build, test summary and archived blms-backend.jar.

What must be visible Use the same build number in all related screenshots.

[Insert screenshot W07-S04 here]

Figure W07-S04 Successful build and archive. Capture date _____ Build or revision _____

Week 8

Pipeline as Code and Server Deployment

Purpose of the milestone

A Jenkinsfile puts the delivery procedure beside the application code. Changes to build or deployment behaviour can then be reviewed as ordinary repository changes, and a particular commit records the instructions used to produce a release.

The current Jenkinsfile includes checkout, backend tests and packaging, frontend tests and build, image building, publishing, deployment verification and browser testing. The backend is an executable Spring Boot JAR with embedded Tomcat, while Nginx serves the built React application. A browser URL alone is not enough deployment evidence; it should be tied to a build and version.

Assignment requirement

Create a Jenkinsfile with checkout, build, package and deploy stages, deploy through Tomcat or Nginx and parameterise an environment setting.

Expected deliverables

- Express the delivery stages in version-controlled code.
- Make a failed command visible as a failed pipeline stage.
- Deploy the application through the selected Tomcat and Nginx architecture.
- Move environment-specific values out of hardcoded application configuration.

Connection to the project

Uses the Jenkins agent and CI job from Week 7. Establishes deployment configuration that later Docker milestones expand.

Week 8

Technical approach and design decisions

Database settings come from DB_URL, DB_USERNAME and DB_PASSWORD for local development, or from the corresponding Spring datasource variables in Compose. The container release uses IMAGE_TAG and REGISTRY_USER. The pipeline binds credentials from Jenkins rather than printing or committing their values.

The pipeline currently deploys the Compose stack before running its browser-test stage. That ordering gives post-deployment validation, not the pre-deployment Selenium gate required later in the brief. It must be described accurately now and corrected before claiming that browser failures prevent promotion to the assessed target.

Decision	Reason or implementation
Pipeline source	Keep stage definitions and failure behaviour in Jenkinsfile.
Backend packaging	Run the executable JAR with embedded Tomcat.
Frontend serving	Use Nginx static files and an /api reverse proxy.
Environment values	Parameterise credentials, release tags and secure-cookie settings.
Traceable deployment	Tie the visible application to a build number and image tag.

Relevant project artefacts

Jenkinsfile

backend/Dockerfile

frontend/Dockerfile

frontend/nginx.conf

docker-compose.yml

.env.example

Use these files to explain the implementation during review. A code view establishes what is defined; the evidence pages separately request the execution record where the brief requires one.

Week 8

Implementation and demonstration procedure

The following sequence is a practical route through this milestone. Check the existing project state first. Historical activities should be supported by genuine history, and any new demonstration should be identified as a new run.

1. Read the Jenkinsfile and explain the purpose and output of each stage.

2. Configure its required tools, agent label and credential identifiers in Jenkins.

3. Prepare a test deployment environment with the correct database credentials.

4. Run the pipeline and inspect each transition from checkout to deployment.

5. Verify the Nginx health endpoint and the proxied application health endpoint.

6. Record which settings are parameterised and identify the browser-gate ordering work still required.

Commands or files to inspect

```
docker compose config --quiet
docker compose up -d --no-build --wait --wait-timeout 180
curl.exe http://localhost:3000/nginx-health
curl.exe http://localhost:3000/api/v1/health
```

Recording the result

Record the source revision, environment and actual outcome alongside the screenshot. If a command fails, retain the error and explain the correction. Replace example host names, tags and paths with the values for the environment being demonstrated. Keep passwords and tokens outside the captured view.

Run context Date _____ Machine or target _____ Revision _____

Week 8

Verification results and discussion

Check	How to verify	Result or evidence state
Stage definition	Compare Jenkinsfile with the visible pipeline graph.	Pipeline source is present; execution screenshot is pending.
Packaged backend	Build the JAR and inspect the archive output.	Local packaging passed.
Frontend serving	Load the app through Nginx rather than the Vite server.	Requires a running Docker or server deployment.
Environment setting	Show variable names and a nonsecret selected image tag.	Compose interpolation was validated locally.
Stage failure	Cause a controlled non-production failure and inspect pipeline status.	Failure behaviour must be demonstrated in Jenkins.

Current outcome

The repository defines an automated container deployment and configurable environment values. Docker engine execution and Jenkins execution were not verified in the current session. The browser-test stage order is an explicit remaining compliance item, not a completed quality gate.

Discussion

Pipeline as code makes a process inspectable, but stage names alone do not prove its behaviour. The order of tests, publishing and deployment determines which failures can actually stop a release.

Completion decision

The milestone can be presented as complete only when its required outputs and the relevant evidence agree. Review the four screenshot requests on the next two pages, fill in the observed identifiers and keep any unresolved requirement visible in the final checklist.

Reviewer comments _____

Week 8

Screenshot evidence A

Insert genuine screenshots in the spaces below. Keep the interface or command output readable and retain the identifiers that connect the image to the activity.

W08-S01 Jenkinsfile stages

How to capture Open the current Jenkinsfile in your editor and show checkout, tests, packaging and deployment definitions.

What must be visible Keep line numbers visible and avoid exposing credential values.

[Insert screenshot W08-S01 here]

Figure W08-S01 Jenkinsfile stages. Capture date _____ Build or revision _____

W08-S02 Pipeline execution graph

How to capture Run the actual job and open its stage view.

What must be visible Show one build number and the outcome of each stage.

[Insert screenshot W08-S02 here]

Figure W08-S02 Pipeline execution graph. Capture date _____ Build or revision _____

Week 8

Screenshot evidence B

Insert genuine screenshots in the spaces below. Keep the interface or command output readable and retain the identifiers that connect the image to the activity.

W08-S03 Application through Nginx

How to capture Open http://localhost:3000 or your real target URL after deployment. Run the two health checks in a terminal.

What must be visible Show a real authenticated application view and the target address.

[Insert screenshot W08-S03 here]

Figure W08-S03 Application through Nginx. Capture date _____ Build or revision _____

W08-S04 Parameterised configuration

How to capture Show .env.example or Compose variable references and the Jenkins credential identifiers.

What must be visible Show names and safe values such as IMAGE_TAG only. Never screenshot a filled private .env file.

[Insert screenshot W08-S04 here]

Figure W08-S04 Parameterised configuration. Capture date _____ Build or revision _____

Week 9

Selenium Test Design and Local Execution

Purpose of the milestone

Browser tests check the application from the user perspective. They can reveal problems that a controller test misses, including a form that sends the wrong request, a status control that is not usable or an apparently successful save that disappears after reload.

The project has fifteen Selenium tests grouped around login, lead creation, search, dashboard and status workflow. During the current validation session all fifteen ran against an isolated H2-backed application and passed with no skips. The creation test uses a unique lead name and checks that the record survives a page reload, reducing the chance of a false pass caused by preloaded sample data.

Assignment requirement

Design three to five critical user journeys with assertions and test data, run WebDriver through Maven and provide failure screenshots.

Expected deliverables

- Select journeys that cover the most important user outcomes.
- Use assertions rather than merely opening pages and clicking controls.
- Use unique synthetic data and explicit waits for asynchronous updates.
- Provide a real failure-capture mechanism and retain its screenshot evidence.

Connection to the project

Requires a working application and meaningful acceptance criteria. Provides the suite that must become a pre-deployment gate in Week 10.

Week 9

Technical approach and design decisions

SeleniumTestBase configures Chrome, a window size and WebDriverWait. BASE_URL, SELENIUM_EMAIL and SELENIUM_PASSWORD let the suite target a real running environment. The readiness probe now uses HTTP/1.1 to avoid a development-server upgrade timeout, and an unavailable application causes failure rather than a skipped suite.

The current base class does not save screenshots automatically on a failed assertion. That part of the Week 9 requirement remains incomplete. The implementation pattern in the appendix explains how to attach a JUnit failure watcher while the driver is still alive, save an image under target/failure-screenshots and archive it in Jenkins.

Decision	Reason or implementation
Five test groups	Login, creation, search, dashboard and status workflow.
Explicit assertions	Check state, visible results and persistence after reload.
Unique records	Use generated identifiers in synthetic lead names.
Environment configuration	Set the URL and credentials for the running target.
Failure evidence	Add automatic screenshots and retain the associated test report.

Relevant project artefacts

backend/src/test/java/com/blms/selenium/SeleniumTestBase.java

backend/src/test/java/com/blms/selenium/CreateLeadTest.java

backend/src/test/java/com/blms/selenium/LoginPageTest.java

backend/pom.xml

Use these files to explain the implementation during review. A code view establishes what is defined; the evidence pages separately request the execution record where the brief requires one.

Week 9

Implementation and demonstration procedure

The following sequence is a practical route through this milestone. Check the existing project state first. Historical activities should be supported by genuine history, and any new demonstration should be identified as a new run.

1. Start the complete test application with a database and valid credentials.

2. Set BASE_URL to its actual URL and set the Selenium account variables without printing passwords.

3. Review the assertions and confirm that test data names are unique.

4. Run the Maven selenium profile and wait for the Surefire summary.

5. Open the generated XML/text reports and check that skipped is zero.

6. Add and verify a failure-screenshot watcher before claiming the screenshot mechanism is complete.

Commands or files to inspect

```
$env:BASE_URL='http://localhost:3000'
$env:SELENIUM_EMAIL='admin@blms.com'
# Set SELENIUM_PASSWORD privately before running tests.
mvn test -Pselenium -B
Get-Content target/surefire-reports/*.txt
```

Recording the result

Record the source revision, environment and actual outcome alongside the screenshot. If a command fails, retain the error and explain the correction. Replace example host names, tags and paths with the values for the environment being demonstrated. Keep passwords and tokens outside the captured view.

Run context Date _____ Machine or target _____ Revision _____

Week 9

Verification results and discussion

Check	How to verify	Result or evidence state
Login	Check page, correct login, incorrect password and logout.	4 tests passed locally.
Lead creation	Check form, creation with reload and blank-name validation.	3 tests passed locally.
Search	Check matching results and clearing the filter.	2 tests passed locally.
Dashboard and status	Check summary elements and progressive status changes.	3 dashboard and 3 workflow tests passed locally.
Failure capture	Deliberately fail a harmless assertion in a test environment.	Automatic screenshot capture is not yet implemented.

Current outcome

The local browser suite passed 15 tests with zero failures, errors or skips. This establishes the exercised UI behaviours on the isolated test database, not a performance benchmark or live PostgreSQL deployment. Failure screenshot automation still needs implementation and proof.

Discussion

A passing test is convincing only when it would fail for the relevant defect. Preloaded data, silent skips and local fallback records can hide a broken backend. Unique data and a reload check make the creation test more useful.

Completion decision

The milestone can be presented as complete only when its required outputs and the relevant evidence agree. Review the four screenshot requests on the next two pages, fill in the observed identifiers and keep any unresolved requirement visible in the final checklist.

Reviewer comments _____

Week 9

Screenshot evidence A

Insert genuine screenshots in the spaces below. Keep the interface or command output readable and retain the identifiers that connect the image to the activity.

W09-S01 Selenium code and assertions

How to capture Open a test class showing a meaningful assertion, unique test data and an explicit wait.

What must be visible For creation, include the reload persistence check.

[Insert screenshot W09-S01 here]

Figure W09-S01 Selenium code and assertions. Capture date _____ Build or revision _____

W09-S02 Local Maven suite result

How to capture Start the application, set BASE_URL and the test credentials, then run mvn test -Pselenium -B.

What must be visible Capture Tests run 15, Failures 0, Errors 0, Skipped 0 from an actual current run.

[Insert screenshot W09-S02 here]

Figure W09-S02 Local Maven suite result. Capture date _____ Build or revision _____

Week 9

Screenshot evidence B

Insert genuine screenshots in the spaces below. Keep the interface or command output readable and retain the identifiers that connect the image to the activity.

W09-S03 Surefire report

How to capture Open backend/target/surefire-reports and a browser test XML or text report.

What must be visible Show the suite name, current timestamp and real counts.

[Insert screenshot W09-S03 here]

Figure W09-S03 Surefire report. Capture date _____ Build or revision _____

W09-S04 Automatic failure screenshot

How to capture After implementing the watcher described in the appendix, introduce one harmless failing assertion and rerun the selected test.

What must be visible Show the generated screenshot file and failing test report, then restore the assertion. Leave this slot pending until it works.

[Insert screenshot W09-S04 here]

Figure W09-S04 Automatic failure screenshot. Capture date _____ Build or revision _____

Week 10

Continuous Testing in Jenkins

Purpose of the milestone

Continuous testing should stop an unsuitable change before it is promoted. The assessment asks for more than publishing a test result: it requires a failed Selenium run to block deployment, followed by a correction and a successful rerun that demonstrate the gate works.

The current pipeline propagates browser-test failures, so a failing suite makes the overall build fail. However, its Browser Tests stage is placed after Deploy and Verify. The target has already been changed at that point. This report records the distinction and provides a staging-first procedure for completing the required pre-deployment gate.

Assignment requirement

Publish Selenium results in Jenkins, make failed tests stop deployment, introduce and correct a deliberate defect, and retain both failed and successful runs.

Expected deliverables

- Run the browser suite on a reachable test or staging environment.
- Publish JUnit XML and retain failure evidence in Jenkins.
- Make the release deployment depend on a successful browser test stage.
- Demonstrate one controlled defect, its correction and the resulting build behaviour.

Connection to the project

Uses the Week 9 suite and Week 8 pipeline. The completed gate should control the container promotion described in Week 12.

Week 10

Technical approach and design decisions

Use a separate temporary stack or staging target to run the browser suite before promotion. Build and test the candidate, start it in the test environment, wait for readiness, run Selenium and publish its reports. Only then publish/promote the candidate and deploy it to the assessed release target. Keep the already deployed stable target unchanged when the browser gate fails.

The exercise can use a deliberately wrong expected label or a small non-production UI defect on a disposable branch. Capture the failing assertion and confirm the promotion stage is skipped. Then correct the actual defect, commit the correction and rerun the pipeline. A manually recoloured stage graph or an old unrelated failure is not valid evidence.

Decision	Reason or implementation
Separate test target	Run the candidate without replacing the release target.
Required browser stage	Do not wrap failures in success-preserving error handling.
Published reports	Use Surefire XML and archive failure screenshots.
Blocked promotion	Confirm deployment was skipped after the controlled failure.
Correction trace	Link the fix commit to a successful rerun of the same test.

Relevant project artefacts

Jenkinsfile

backend/src/test/java/com/blms/selenium

backend/target/surefire-reports

Suggested failure-screenshot watcher in Appendix C

Use these files to explain the implementation during review. A code view establishes what is defined; the evidence pages separately request the execution record where the brief requires one.

Week 10

Implementation and demonstration procedure

The following sequence is a practical route through this milestone. Check the existing project state first. Historical activities should be supported by genuine history, and any new demonstration should be identified as a new run.

1. Adjust the pipeline so the candidate runs in staging before release deployment.

2. Configure BASE_URL and credentials for that staging target.

3. Publish browser XML in an always block so failed runs remain inspectable.

4. Introduce a controlled defect on an isolated branch and run the pipeline.

5. Capture the failing test and the skipped promotion/deployment stage.

6. Correct the defect in source, commit the fix and capture the successful rerun.

Commands or files to inspect

```
mvn test -Pselenium -B
git diff --check
git log -1 --oneline
git show --stat HEAD
```

Recording the result

Record the source revision, environment and actual outcome alongside the screenshot. If a command fails, retain the error and explain the correction. Replace example host names, tags and paths with the values for the environment being demonstrated. Keep passwords and tokens outside the captured view.

Run context Date _____ Machine or target _____ Revision _____

Week 10

Verification results and discussion

Check	How to verify	Result or evidence state
Report publication	Open Jenkins test results after both pass and fail.	Jenkinsfile defines publication; live proof is pending.
Real failure	Use a deliberately incorrect expected value in staging.	Must produce a genuine failed assertion.
Deployment blocked	Inspect the stage graph after the failed test.	Current stage order does not satisfy this condition.
Defect corrected	Show the source diff and correction commit.	Requires an actual controlled exercise.
Successful rerun	Match the new passing run to the fix revision.	No Jenkins execution was verified in this session.

Current outcome

Local Selenium execution is verified, and browser failures are no longer suppressed. Pre-deployment blocking remains incomplete because of the stage order. The failed-build, correction-commit and successful-rerun screenshots must come from a genuine staging exercise.

Discussion

A quality gate is defined by what it prevents. Failing the build after a release has already reached its target may detect a problem, but it cannot be presented as proof that the deployment was blocked.

Completion decision

The milestone can be presented as complete only when its required outputs and the relevant evidence agree. Review the four screenshot requests on the next two pages, fill in the observed identifiers and keep any unresolved requirement visible in the final checklist.

Reviewer comments _____

Week 10

Screenshot evidence A

Insert genuine screenshots in the spaces below. Keep the interface or command output readable and retain the identifiers that connect the image to the activity.

W10-S01 Published Jenkins tests

How to capture Run the Selenium stage and open the Jenkins test-result page.

What must be visible Show suite names, pass/fail counts, build number and checked-out commit.

[Insert screenshot W10-S01 here]

Figure W10-S01 Published Jenkins tests. Capture date _____ Build or revision _____

W10-S02 Failed test blocks deployment

How to capture Run the controlled defect after moving browser tests before release promotion. Open the stage graph.

What must be visible Show the failed browser stage and the release deployment stage marked skipped or not executed.

[Insert screenshot W10-S02 here]

Figure W10-S02 Failed test blocks deployment. Capture date _____ Build or revision _____

Week 10

Screenshot evidence B

Insert genuine screenshots in the spaces below. Keep the interface or command output readable and retain the identifiers that connect the image to the activity.

W10-S03 Defect correction commit

How to capture Open the real correction diff and its Git commit or PR.

What must be visible Show the changed assertion or application logic and explain the cause in the caption.

[Insert screenshot W10-S03 here]

Figure W10-S03 Defect correction commit. Capture date _____ Build or revision _____

W10-S04 Successful rerun

How to capture Rerun the corrected revision and open the successful stage graph and test summary.

What must be visible Use the same journey as the failure demonstration, with deployment occurring only after tests pass.

[Insert screenshot W10-S04 here]

Figure W10-S04 Successful rerun. Capture date _____ Build or revision _____

Week 11

Docker Image and Container Lifecycle

Purpose of the milestone

Containers package the application runtime so the same built output can be started consistently on another suitable host. BLMS uses separate images for the Spring Boot backend and Nginx frontend, together with a PostgreSQL service and a persistent database volume.

Both application Dockerfiles use build stages. Maven compiles and packages the backend before its JAR is copied into a Java runtime image. Node installs frontend dependencies and builds static assets before they are copied into Nginx. The backend runtime uses a dedicated unprivileged user, and curl is explicitly installed for its health check.

Assignment requirement

Create Dockerfiles, build and tag images, run containers with mapped ports, inspect logs and demonstrate stop, restart and removal.

Expected deliverables

- Describe how the two application images are assembled.
- Build versioned images from the current source.
- Run the stack and verify mapped ports, logs and health.
- Demonstrate the container lifecycle without deleting persistent application data.

Connection to the project

Uses the packaged application and deployment configuration. Supplies the images that the Jenkins release pipeline should build and publish.

Week 11

Technical approach and design decisions

Compose waits for PostgreSQL health before starting the backend, and for backend health before the frontend. The database uses postgres_data so stopping or recreating a container does not automatically discard its records. Host bindings for PostgreSQL and the backend use loopback, while services communicate through the internal Compose network.

The distinction between an image, a container and a volume matters during the lifecycle demonstration. Removing a stopped disposable application container does not remove its image or database volume. Avoid docker compose down -v in the normal exercise because it removes the named database volume and changes the persistence behaviour being demonstrated.

Decision	Reason or implementation
Backend build stage	Use Maven to produce the executable JAR.
Frontend build stage	Use npm ci and Vite before copying assets into Nginx.
Runtime user	Run the backend as blmsuser.
Readiness	Use database-aware backend health and Nginx liveness checks.
Persistent volume	Retain PostgreSQL data across container recreation.

Relevant project artefacts

backend/Dockerfile

frontend/Dockerfile

frontend/nginx.conf

docker-compose.yml

.env.example

Use these files to explain the implementation during review. A code view establishes what is defined; the evidence pages separately request the execution record where the brief requires one.

Week 11

Implementation and demonstration procedure

The following sequence is a practical route through this milestone. Check the existing project state first. Historical activities should be supported by genuine history, and any new demonstration should be identified as a new run.

1. Start Docker Desktop with its Linux engine and confirm Docker and Compose versions.

2. Copy .env.example to a private .env and set the database and bootstrap account values.

3. Select a safe demonstration image tag, then build and start the stack.

4. Inspect container status, mapped ports, logs and health.

5. Stop and restart the frontend to demonstrate its lifecycle without changing database data.

6. Remove only a stopped disposable frontend container and recreate it through Compose.

Commands or files to inspect

```
docker version
docker compose version
$env:IMAGE_TAG='demo-week11'
docker compose build
docker compose up -d --wait --wait-timeout 180
docker compose ps
docker compose logs --tail 40 backend frontend
```

Recording the result

Record the source revision, environment and actual outcome alongside the screenshot. If a command fails, retain the error and explain the correction. Replace example host names, tags and paths with the values for the environment being demonstrated. Keep passwords and tokens outside the captured view.

Run context Date _____ Machine or target _____ Revision _____

Week 11

Verification results and discussion

Check	How to verify	Result or evidence state
Image build	Inspect the build log and versioned image list.	Dockerfiles exist; Docker engine execution is pending.
Running services	Check db, backend and frontend health.	Compose configuration validates locally.
Port mapping	Open the frontend and test health through Nginx.	Requires the actual running stack.
Lifecycle	Stop, start, inspect and recreate the frontend.	Capture commands and observed transitions.
Persistence	Create a test lead, recreate only application containers and reload.	Do not delete the database volume during this check.

Current outcome

Container definitions and variable interpolation were validated. The Docker engine was unavailable during the project assessment, so no running-container, image-build or lifecycle claim is made here. The four evidence slots describe the remaining demonstration.

Discussion

Reproducibility depends on both the image and its runtime configuration. A correct image can still fail when its environment, network, port or persistent storage is wrong.

Completion decision

The milestone can be presented as complete only when its required outputs and the relevant evidence agree. Review the four screenshot requests on the next two pages, fill in the observed identifiers and keep any unresolved requirement visible in the final checklist.

Reviewer comments _____

Week 11

Screenshot evidence A

Insert genuine screenshots in the spaces below. Keep the interface or command output readable and retain the identifiers that connect the image to the activity.

W11-S01 Dockerfiles and build

How to capture Show the multi-stage Dockerfile and run docker compose build with the demonstration tag configured.

What must be visible Capture the successful build ending and the source revision used.

[Insert screenshot W11-S01 here]

Figure W11-S01 Dockerfiles and build. Capture date _____ Build or revision _____

W11-S02 Image details

How to capture Run docker image ls and docker image inspect on the specific backend and frontend tags.

What must be visible Show the full tag and image identifier. Avoid unrelated private registry details.

[Insert screenshot W11-S02 here]

Figure W11-S02 Image details. Capture date _____ Build or revision _____

Week 11

Screenshot evidence B

Insert genuine screenshots in the spaces below. Keep the interface or command output readable and retain the identifiers that connect the image to the activity.

W11-S03 Running stack and logs

How to capture Run docker compose ps and docker compose logs --tail 40 backend frontend. Open the application at localhost:3000.

What must be visible Show healthy services, mapped ports and actual startup logs.

[Insert screenshot W11-S03 here]

Figure W11-S03 Running stack and logs. Capture date _____ Build or revision _____

W11-S04 Container lifecycle

How to capture Capture docker compose stop frontend, start frontend, stop frontend, rm -f frontend and up -d frontend in order.

What must be visible Show the real status changes. Keep the database volume and its test records intact.

[Insert screenshot W11-S04 here]

Figure W11-S04 Container lifecycle. Capture date _____ Build or revision _____

Week 12

Jenkins Docker Continuous Deployment

Purpose of the milestone

A release becomes traceable when its commit, build, image tag and running container can be linked. Rebuilding an unlabelled latest image makes that chain difficult to prove, particularly when a rollback must select a previous release.

The current pipeline uses IMAGE_TAG=build-${BUILD_NUMBER} and builds both application images under the configured registry namespace. Compose now interpolates IMAGE_TAG and REGISTRY_USER, so an explicitly selected release is used by deployment and rollback. Publishing and health-check failures cause the pipeline to fail rather than being ignored.

Assignment requirement

Build a versioned image in Jenkins, publish it to a registry and deploy a fresh container automatically after successful tests.

Expected deliverables

- Build uniquely tagged application images from the checked-out revision.
- Authenticate to the registry through Jenkins credentials.
- Publish and deploy the selected images without silently rebuilding a different release.
- Show a continuous chain from commit to build to running container.

Connection to the project

Uses the Docker images and lifecycle knowledge from Week 11 and the required browser gate from Week 10.

Week 12

Technical approach and design decisions

Jenkins expects docker-hub, blms-db-password and blms-admin-password credential entries. The registry password is supplied to docker login through standard input. Database and bootstrap values are bound only around the commands that need them. A screenshot should show credential identifiers and masked bindings, never the stored secret.

The current Browser Tests stage still follows deployment. To meet the brief exactly, the staging-first gate from Week 10 must be completed and placed before release promotion. Registry publishing can then follow the successful candidate test, and the release target can use the same tested image rather than rebuilding from changed source.

Decision	Reason or implementation
Release identifier	Use build-number tags and record the source commit.
Credential binding	Keep registry and database secrets outside source control.
No silent fallback	Fail when image publishing or health checks fail.
Exact image selection	Use Compose IMAGE_TAG interpolation and --no-build.
Promotion condition	Require staging browser tests before release deployment.

Relevant project artefacts

Jenkinsfile

docker-compose.yml

backend/Dockerfile

frontend/Dockerfile

ansible/deploy-playbook.yml

Use these files to explain the implementation during review. A code view establishes what is defined; the evidence pages separately request the execution record where the brief requires one.

Week 12

Implementation and demonstration procedure

The following sequence is a practical route through this milestone. Check the existing project state first. Historical activities should be supported by genuine history, and any new demonstration should be identified as a new run.

1. Configure the registry account and Jenkins credential identifiers.

2. Complete the Week 10 test gate before presenting this as a compliant release pipeline.

3. Run a build from a reviewed commit and note its build-number tag.

4. Verify that both tags appear in the registry.

5. Inspect the deployment stage and the running container image reference.

6. Create or edit a unique test lead after deployment and capture the health response and visible application.

Commands or files to inspect

```
docker compose config --images
docker image ls
docker compose ps
docker inspect blms-backend-container --format "{{.Config.Image}}"
curl.exe http://localhost:3000/api/v1/health
```

Recording the result

Record the source revision, environment and actual outcome alongside the screenshot. If a command fails, retain the error and explain the correction. Replace example host names, tags and paths with the values for the environment being demonstrated. Keep passwords and tokens outside the captured view.

Run context Date _____ Machine or target _____ Revision _____

Week 12

Verification results and discussion

Check	How to verify	Result or evidence state
Versioned images	Compare Jenkins build number and registry tag.	Tag construction is implemented; publish evidence is pending.
Registry push	Open the actual repository tag list.	Requires a successful authorised registry push.
Exact deployment	Inspect the running container image reference.	Compose interpolation was checked with a sample tag.
Failed-test blocking	Fail the staging suite and check release promotion.	Not satisfied by the current stage ordering.
Fresh container	Compare container creation time and selected release.	Requires a live Jenkins/Docker demonstration.

Current outcome

The source supports versioned build, publishing and selected-image deployment. The configuration no longer ignores registry or health-check failures. End-to-end registry and container evidence is still required, and the pre-deployment browser gate remains an open compliance item.

Discussion

The useful unit of release evidence is the entire chain, not an isolated green stage. The commit, successful tests, registry tag and running image should all describe the same candidate.

Completion decision

The milestone can be presented as complete only when its required outputs and the relevant evidence agree. Review the four screenshot requests on the next two pages, fill in the observed identifiers and keep any unresolved requirement visible in the final checklist.

Reviewer comments _____

Week 12

Screenshot evidence A

Insert genuine screenshots in the spaces below. Keep the interface or command output readable and retain the identifiers that connect the image to the activity.

W12-S01 Versioned Jenkins image build

How to capture Run the pipeline from a known revision and capture the Docker build stage.

What must be visible Show build number, selected IMAGE_TAG and both application image names.

[Insert screenshot W12-S01 here]

Figure W12-S01 Versioned Jenkins image build. Capture date _____ Build or revision _____

W12-S02 Registry publication

How to capture Open Docker Hub or your actual registry and show the two tags produced by that build.

What must be visible Keep account tokens hidden. Do not use an unrelated old tag as current evidence.

[Insert screenshot W12-S02 here]

Figure W12-S02 Registry publication. Capture date _____ Build or revision _____

Week 12

Screenshot evidence B

Insert genuine screenshots in the spaces below. Keep the interface or command output readable and retain the identifiers that connect the image to the activity.

W12-S03 Running selected image

How to capture Run docker inspect blms-backend-container --format "{{.Config.Image}}" and show docker compose ps.

What must be visible The tag must match the tested Jenkins build.

[Insert screenshot W12-S03 here]

Figure W12-S03 Running selected image. Capture date _____ Build or revision _____

W12-S04 Commit to container run

How to capture Capture the pipeline graph and the newly deployed application with a unique test record.

What must be visible After completing the staging gate, show browser tests preceding release deployment.

[Insert screenshot W12-S04 here]

Figure W12-S04 Commit to container run. Capture date _____ Build or revision _____

Week 13

Configuration Management Script

Purpose of the milestone

Configuration management records the host requirements that are otherwise easy to miss during manual setup. BLMS needs packages, an application user, predictable directories, environment configuration, service management and network access before deployment can be repeated reliably.

The repository uses Ansible and targets an Ubuntu node. Its provisioning playbook installs the required packages, creates a service user and application folders, writes a protected environment file and starts Docker. The report uses Ansible consistently; it does not claim that a separate Puppet implementation exists.

Assignment requirement

List server prerequisites and provide an Ansible inventory and playbook or equivalent Puppet configuration, with a first execution log.

Expected deliverables

- Define the required host state before application deployment.
- Keep machine addresses and access configuration in inventory.
- Use declarative modules for repeatable prerequisite setup.
- Capture the first real execution and distinguish it from a syntax check.

Connection to the project

Uses the deployment prerequisites identified in the Docker work. Establishes the clean target needed for Week 14 reliability checks.

Week 13

Technical approach and design decisions

Ansible should run from a Linux control environment such as WSL or a Linux VM. The target should be a disposable or approved Ubuntu node. The supplied inventory contains an example address and an SSH key path; these must be replaced with the actual target before execution. Use --limit so a demonstration cannot accidentally apply to every listed host.

The playbook accepts a database password through Ansible variables or the environment and writes its configuration with restrictive file permissions. Package availability, privilege escalation and the Docker service are target-dependent. A successful YAML parse only establishes syntax, not connectivity, package installation or a secure operational state.

Decision	Reason or implementation
Control environment	Use WSL or Linux for the Ansible commands.
Target selection	Replace the example host and use --limit.
Service identity	Create blmsuser and blmsgroup on the target.
Directory state	Create /opt/blms configuration, deployment and log directories.
Private configuration	Supply secrets privately and avoid logging their values.

Relevant project artefacts

ansible/inventory.ini

ansible/ansible.cfg

ansible/playbook.yml

ansible/deploy-playbook.yml

Use these files to explain the implementation during review. A code view establishes what is defined; the evidence pages separately request the execution record where the brief requires one.

Week 13

Implementation and demonstration procedure

The following sequence is a practical route through this milestone. Check the existing project state first. Historical activities should be supported by genuine history, and any new demonstration should be identified as a new run.

1. Prepare a WSL or Linux control machine and an approved Ubuntu target.
2. Update the inventory with the real IP or DNS name, SSH user and key location.
3. Check SSH access and become/sudo permissions before provisioning.
4. Run the inventory listing, connectivity check and playbook syntax check.
5. Supply private variables securely, then run the provisioning playbook with --limit.
6. Inspect package, user, directory and service state on the target and save the genuine recap.

Commands or files to inspect

```
cd ansible
ansible-inventory --graph
ansible blms_servers --limit app-server-01 -m ping
ansible-playbook playbook.yml --syntax-check
ansible-playbook playbook.yml --limit app-server-01 --ask-become-pass
```

Recording the result

Record the source revision, environment and actual outcome alongside the screenshot. If a command fails, retain the error and explain the correction. Replace example host names, tags and paths with the values for the environment being demonstrated. Keep passwords and tokens outside the captured view.

Run context Date _____ Machine or target _____ Revision _____

Week 13

Verification results and discussion

Check	How to verify	Result or evidence state
Inventory	Check target address and group membership.	Example inventory exists; real target configuration is pending.
Syntax	Parse YAML and run Ansible syntax validation.	YAML parsing passed; native Ansible execution was not run.
Packages	Check the required package list on the target.	Needs a first execution log.
Files and user	Inspect /opt/blms and the service identity.	Playbook definitions exist; target-state proof is pending.
Docker service	Check enabled/running state and docker info.	Requires a reachable target with Docker installed.

Current outcome

The inventory and playbooks are available and their YAML parses. This session did not provision a target node or verify an Ansible execution. The first-run recap and resulting machine state remain evidence to collect.

Discussion

A playbook is a specification of desired machine state. Its value is demonstrated by applying it to a real target and inspecting what changed, including any assumptions about packages, privileges or networking.

Completion decision

The milestone can be presented as complete only when its required outputs and the relevant evidence agree. Review the four screenshot requests on the next two pages, fill in the observed identifiers and keep any unresolved requirement visible in the final checklist.

Reviewer comments _____

Week 13

Screenshot evidence A

Insert genuine screenshots in the spaces below. Keep the interface or command output readable and retain the identifiers that connect the image to the activity.

W13-S01 Prerequisites and inventory

How to capture Open inventory.ini and the package/user/folder sections of playbook.yml.

What must be visible Replace the example IP first. Hide private SSH key contents and secret values.

[Insert screenshot W13-S01 here]

Figure W13-S01 Prerequisites and inventory. Capture date _____ Build or revision _____

W13-S02 Control machine connectivity

How to capture From WSL or Linux run ansible-inventory --graph and the limited ping command.

What must be visible Show the selected target and a genuine SUCCESS response.

[Insert screenshot W13-S02 here]

Figure W13-S02 Control machine connectivity. Capture date _____ Build or revision _____

Week 13

Screenshot evidence B

Insert genuine screenshots in the spaces below. Keep the interface or command output readable and retain the identifiers that connect the image to the activity.

W13-S03 First provisioning run

How to capture Run the provisioning playbook with --limit and capture the task output and PLAY RECAP.

What must be visible Show actual ok, changed, failed and unreachable counts. Do not type a recap manually.

[Insert screenshot W13-S03 here]

Figure W13-S03 First provisioning run. Capture date _____ Build or revision _____

W13-S04 Provisioned host state

How to capture On the target show id blmsuser, the /opt/blms directory structure and systemctl status docker.

What must be visible Do not print /opt/blms/config/.env because it contains private configuration.

[Insert screenshot W13-S04 here]

Figure W13-S04 Provisioned host state. Capture date _____ Build or revision _____

Week 14

Automated Provisioning and Reliability Validation

Purpose of the milestone

Reliability work checks how the application behaves beyond its first successful start. A target should be reproducible, repeated configuration should not cause unnecessary changes, health should reflect dependencies, and rollback should select a known earlier release.

BLMS has separate provisioning, deployment and rollback playbooks. Deployment pulls selected images and waits for service health. Rollback now passes the selected version through Compose interpolation and pulls the earlier images before recreating application containers. This is a practical correction to a configuration that previously set a tag variable without using it.

Assignment requirement

Provision a clean environment, deploy the application, rerun automation for idempotency, check health and demonstrate rollback to a previous stable release.

Expected deliverables

- Demonstrate provisioning on a clean or clearly reset target.
- Run prerequisite configuration twice and interpret the second recap.
- Check backend database connectivity and frontend liveness.
- Recreate the application at a real previous image tag and verify recovery.

Connection to the project

Requires the Week 13 target and at least two compatible published releases from the container pipeline.

Week 14

Technical approach and design decisions

Idempotency should be assessed on the provisioning playbook and the actual resulting host state. A second run should normally have no unnecessary changes. The deployment playbook includes commands and a changed_when expression that inspects stdout; some Compose progress is written elsewhere. Its reported changed count alone is therefore not enough proof of whether containers were recreated.

Rollback is a container replacement, so a brief interruption is expected. It does not reverse database schema changes or restore deleted data. Both the previous image and database compatibility must be checked before the demonstration. Keep the database volume and verify that a unique lead remains after the application containers are recreated.

Decision	Reason or implementation
Clean target	Record the machine identity and initial state.
Repeatability	Compare first and second provisioning runs.
Dependency health	Backend health performs SELECT 1 against its database.
Previous release	Use a real published version rather than an assumed tag.
Rollback boundary	Application image rollback does not roll back the database.

Relevant project artefacts

ansible/playbook.yml

ansible/deploy-playbook.yml

ansible/rollback-playbook.yml

docker-compose.yml

backend/src/main/java/com/blms/controller/HealthController.java

Use these files to explain the implementation during review. A code view establishes what is defined; the evidence pages separately request the execution record where the brief requires one.

Week 14

Implementation and demonstration procedure

The following sequence is a practical route through this milestone. Check the existing project state first. Historical activities should be supported by genuine history, and any new demonstration should be identified as a new run.

1. Provision the approved clean Ubuntu node and retain its first execution recap.

2. Rerun the same provisioning playbook without changing variables.

3. Investigate any unexpected changed tasks instead of claiming idempotency automatically.

4. Deploy a real published candidate tag and create a unique synthetic lead.

5. Run rollback with an actual previous compatible tag and inspect the running image.

6. Check both health endpoints and confirm the stored lead still exists after recovery.

Commands or files to inspect

```
ansible-playbook playbook.yml --limit app-server-01
ansible-playbook playbook.yml --limit app-server-01
ansible-playbook deploy-playbook.yml --limit app-server-01 -e image_tag=CANDIDATE_TAG
ansible-playbook rollback-playbook.yml --limit app-server-01 -e rollback_version=PREVIOUS_TAG
curl -f http://localhost:8081/api/v1/health
```

Recording the result

Record the source revision, environment and actual outcome alongside the screenshot. If a command fails, retain the error and explain the correction. Replace example host names, tags and paths with the values for the environment being demonstrated. Keep passwords and tokens outside the captured view.

Run context Date _____ Machine or target _____ Revision _____

Week 14

Verification results and discussion

Check	How to verify	Result or evidence state
Clean provisioning	Record target identity and first-run result.	No target was provisioned in this session.
Second run	Inspect changed tasks and recap.	Needs a real rerun; source alone is insufficient.
Selected version	Compare requested tag with docker inspect.	Variable interpolation passed a local configuration check.
Health	Check database-aware backend and frontend endpoints.	Implemented; target runtime proof is pending.
Recovery and data	Inspect the earlier image and reload a known test lead.	Requires a real compatible previous release.

Current outcome

The source supports selected-image deployment and rollback, and configuration checks confirm the image tag is interpolated. Clean-node provisioning, idempotency and actual recovery remain unverified until executed on the target. The report does not describe the procedure as zero downtime.

Discussion

A recovery claim should identify what was restored and what was left untouched. Replacing an application image is useful, but it cannot establish database recovery or a measured availability guarantee.

Completion decision

The milestone can be presented as complete only when its required outputs and the relevant evidence agree. Review the four screenshot requests on the next two pages, fill in the observed identifiers and keep any unresolved requirement visible in the final checklist.

Reviewer comments _____

Week 14

Screenshot evidence A

Insert genuine screenshots in the spaces below. Keep the interface or command output readable and retain the identifiers that connect the image to the activity.

W14-S01 Clean target and first run

How to capture Capture the target hostname/OS and the first provisioning PLAY RECAP.

What must be visible Show that the chosen node is the one used in later deployment screenshots.

[Insert screenshot W14-S01 here]

Figure W14-S01 Clean target and first run. Capture date _____ Build or revision _____

W14-S02 Idempotent rerun

How to capture Rerun the same provisioning playbook and capture its recap and any changed task.

What must be visible Claim changed=0 only if the real output shows it. Explain legitimate changes if present.

[Insert screenshot W14-S02 here]

Figure W14-S02 Idempotent rerun. Capture date _____ Build or revision _____

Week 14

Screenshot evidence B

Insert genuine screenshots in the spaces below. Keep the interface or command output readable and retain the identifiers that connect the image to the activity.

W14-S03 Deployment and health

How to capture Deploy a real candidate tag, then show container health and a unique saved lead.

What must be visible Capture the image tag and both health endpoints.

[Insert screenshot W14-S03 here]

Figure W14-S03 Deployment and health. Capture date _____ Build or revision _____

W14-S04 Rollback and recovery

How to capture Run the rollback playbook with a real earlier tag and inspect the resulting image reference. Reload the unique lead.

What must be visible Show the previous tag, recovery health and preserved data. Do not claim database rollback.

[Insert screenshot W14-S04 here]

Figure W14-S04 Rollback and recovery. Capture date _____ Build or revision _____

Week 15

Final Release Documentation and Viva

Purpose of the milestone

The final milestone brings the application and its delivery evidence into one consistent account. A reader should be able to understand the problem, identify the tested source revision, follow how a release was built and deployed, and see which requirements still need work.

The local verification record is strong for the application: 31 backend tests, 15 browser tests and five frontend API tests passed with no skips. Maven packaging and the Vite production build also passed. Those results should be presented with their environment limits. The browser run used an isolated H2 test database, and no live Jenkins, registry, Docker or Ansible deployment was verified in that session.

Assignment requirement

Demonstrate the complete workflow and complete technical documentation, architecture, troubleshooting, limitations, enhancement plans, presentation and viva preparation.

Expected deliverables

- Assemble the weekly work and its evidence into one navigable report.
- Trace a final reviewed commit through tests and deployment.
- Prepare a short demonstration and clear answers about design decisions.
- State limitations and unfinished evidence without overstating completion.

Connection to the project

Uses the outputs of all previous weeks. Final sign-off depends on the remaining requirement and evidence checklist, not on report length alone.

Week 15

Technical approach and design decisions

The final evidence chain should connect the repository revision, Jenkins build number, browser quality-gate result, image tag, running container and provisioned host. Screenshots should use matching identifiers and be captured from the actual tools. A screenshot of source code is evidence of implementation, while a running-service screenshot is evidence of execution.

This report includes current architecture, runbooks, a requirements comparison, screenshot instructions and a final checklist. Student details come from the supplied reports, with the institution and professor supplied in the conversation. Signatures, approval dates, presentation delivery and final evaluator acceptance remain fields to complete from actual events.

Decision	Reason or implementation
One baseline	Use the same reviewed revision throughout the final demonstration.
Current documentation	Use PostgreSQL, port 8081, server sessions and versioned images.
Evidence separation	Distinguish code, local execution and target deployment.
Viva preparation	Explain decisions using actual files and observed behaviour.
Honest limitations	Retain open requirement and delivery gaps in the final summary.

Relevant project artefacts

README.md

CHANGELOG.md

docs/BACKLOG.md

Jenkinsfile

This consolidated report and its screenshot register

Use these files to explain the implementation during review. A code view establishes what is defined; the evidence pages separately request the execution record where the brief requires one.

Week 15

Implementation and demonstration procedure

The following sequence is a practical route through this milestone. Check the existing project state first. Historical activities should be supported by genuine history, and any new demonstration should be identified as a new run.

1. Review and commit the intended source changes through the agreed repository process.

2. Run the final tests and note the source revision and execution environment.

3. Complete the pre-deployment browser gate and failure-screenshot mechanism.

4. Capture the registry, container, provisioning and rollback evidence on the real target.

5. Insert screenshots in the named slots and update each caption with actual build or version identifiers.

6. Update the Word contents table, review page numbering and prepare the live demonstration and presentation.

Commands or files to inspect

```
git status --short
git rev-parse HEAD
mvn package -B
npm test
npm run build
mvn test -Pselenium -B
```

Recording the result

Record the source revision, environment and actual outcome alongside the screenshot. If a command fails, retain the error and explain the correction. Replace example host names, tags and paths with the values for the environment being demonstrated. Keep passwords and tokens outside the captured view.

Run context Date _____ Machine or target _____ Revision _____

Week 15

Verification results and discussion

Check	How to verify	Result or evidence state
Application regression	Review backend, browser and frontend reports.	31 + 15 + 5 local tests passed.
Build outputs	Inspect backend JAR and frontend dist output.	Both builds passed locally.
End-to-end release	Trace one commit to a tested and running selected image.	Needs live Jenkins/registry/container evidence.
Operations	Show provisioning rerun, health and rollback.	Needs target execution evidence.
Submission completeness	Check screenshots, captions, sources and open gaps.	Complete the final checklist before signing or submitting.

Current outcome

The consolidated report covers all fifteen tasks and preserves the meaning of the earlier reports while correcting technical inconsistencies. It is an evidence-completion copy: screenshot spaces, approval details and the recorded open gaps must be completed honestly before final submission.

Discussion

The final demonstration is strongest when each claim can be checked against a specific artefact. Being able to explain an unfinished requirement is more defensible than presenting an unsupported claim of complete delivery.

Future enhancement plan

First complete ownership, actor auditing and strict status transitions, then finish the staging test gate and automatic failure screenshots. The next release should add schema migrations, tested database backups and pagination. After those foundations are verified, consider notifications and external lead sources. Keep mobile and multi-branch support outside the current MVP and estimate them separately.

Reviewer comments _____

Week 15

Screenshot evidence A

Insert genuine screenshots in the spaces below. Keep the interface or command output readable and retain the identifiers that connect the image to the activity.

W15-S01 Final repository baseline

How to capture Show the final reviewed commit or release tag and a clean or explicitly explained working tree.

What must be visible Include the revision that the final tests and release actually used.

[Insert screenshot W15-S01 here]

Figure W15-S01 Final repository baseline. Capture date _____ Build or revision _____

W15-S02 End to end pipeline

How to capture Capture a real pipeline graph showing the required browser gate before release deployment.

What must be visible Use one identifiable build and include the test summary.

[Insert screenshot W15-S02 here]

Figure W15-S02 End to end pipeline. Capture date _____ Build or revision _____

Week 15

Screenshot evidence B

Insert genuine screenshots in the spaces below. Keep the interface or command output readable and retain the identifiers that connect the image to the activity.

W15-S03 Live final application

How to capture Show the running target URL, role-appropriate lead workflow and summary dashboard.

What must be visible Use synthetic data and make the deployed image/build identity traceable.

[Insert screenshot W15-S03 here]

Figure W15-S03 Live final application. Capture date _____ Build or revision _____

W15-S04 Provisioning and final demonstration

How to capture Capture the final host health/rollback proof or a genuine demonstration overview.

What must be visible Record the actual demonstration date and leave professor approval unsigned until it is given.

[Insert screenshot W15-S04 here]

Figure W15-S04 Provisioning and final demonstration. Capture date _____ Build or revision _____

Appendix A

Local development runbook

Use this sequence for a normal PostgreSQL development run on Windows. The project directory is C:\Users\Eshan\Downloads\broker_lead_management. Open separate terminals for the backend and frontend. Existing database volumes or installations retain their original user and password; match those credentials instead of expecting an environment variable to reset them.

Backend terminal

```
Set-Location 'C:\Users\Eshan\Downloads\broker_lead_management\backend'
$env:DB_URL='jdbc:postgresql://localhost:5432/blms'
$env:DB_USERNAME='YOUR_DATABASE_USER'
# Set DB_PASSWORD privately in this terminal.
# For a first admin only, set BLMS_ADMIN_EMAIL and BLMS_ADMIN_PASSWORD privately.
mvn spring-boot:run
```

The bootstrap administrator is created only when its email does not already exist. Bootstrap does not reset an existing account password or change its role. Demo account seeding is disabled unless BLMS_DEMO_DATA is explicitly enabled on an empty database.

Frontend terminal

```
Set-Location 'C:\Users\Eshan\Downloads\broker_lead_management\frontend'
npm ci
npm run dev
```

Verification

```
curl.exe http://localhost:8081/api/v1/health
# Open http://localhost:5173 in the browser.
```

Sign in, create a uniquely named test lead, reload and inspect the dashboard. Capture the frontend address and the health output. When serving over HTTPS, configure secure session cookies. Do not claim that local HTTP is an HTTPS deployment.

Appendix B

Docker demonstration runbook

Use a disposable demonstration environment with synthetic data. Start Docker Desktop and its Linux engine before running these commands. A successful docker compose config is not evidence that the engine or containers are running.

1. Open the repository root and copy .env.example to .env if a private .env does not already exist.

2. Fill in DB_PASSWORD and the intended administrator bootstrap details privately. Keep the file untracked.

3. Choose a descriptive demonstration image tag. Build and start the three services.

4. Wait for health and open localhost:3000. Create a unique test lead.

5. Inspect logs and image references. Then demonstrate lifecycle operations on the frontend only.

6. Recreate the frontend and confirm the application is usable. Keep the PostgreSQL volume intact.

```
$env:IMAGE_TAG='demo-week11'
docker compose config --quiet
docker compose build
docker compose up -d --wait --wait-timeout 180
docker compose ps
docker compose stop frontend
docker compose start frontend
docker compose stop frontend
docker compose rm -f frontend
docker compose up -d frontend
```

For the evidence, use docker image ls, docker compose logs --tail 40 backend frontend and docker inspect on the selected image. Avoid printing a fully resolved Compose configuration because it can include secrets. Do not use down -v when demonstrating persistence.

Appendix C

Selenium execution and failure capture

Run the suite only after the application is healthy and the chosen account can sign in. Set BASE_URL, SELENIUM_EMAIL and SELENIUM_PASSWORD for that environment. The local verification used an isolated H2 application; an ordinary assessment run can use the configured PostgreSQL stack.

```
mvn test -Pselenium -B
Get-Content target/surefire-reports/com.blms.selenium.*.txt
```

Automatic failure screenshots are still missing from the source. One implementation route is a JUnit invocation interceptor that captures an exception before the driver is closed. Register the extension in SeleniumTestBase and save under target/failure-screenshots. The following is an implementation pattern to adapt and test, not a claim that the mechanism is already installed.

```
@RegisterExtension
final InvocationInterceptor failureCapture = new InvocationInterceptor() {
  @Override public void interceptTestMethod(Invocation<Void> call,
     ReflectiveInvocationContext<Method> method, ExtensionContext context)
     throws Throwable {
    try { call.proceed(); } catch (Throwable failure) {
     try {
       Path folder = Path.of("target", "failure-screenshots");
       Files.createDirectories(folder);
       if (driver instanceof TakesScreenshot camera) {
         String name = context.getRequiredTestMethod().getName();
         Files.write(folder.resolve(name + ".png"),
             camera.getScreenshotAs(OutputType.BYTES));
       }
     } catch (Exception captureError) { failure.addSuppressed(captureError); }
     throw failure;
    }
  }
};
```

Add the relevant JUnit extension, reflection, java.nio.file and Selenium imports. This pattern captures failures in the test method; also consider setup failures for a complete mechanism. Archive the folder in Jenkins with allowEmptyArchive only for runs that have no failures. Verify it with a deliberately failing test before submission.

Appendix D

Jenkins gate and controlled defect exercise

The brief requires browser failures to stop release deployment. The current Jenkinsfile runs Browser Tests after Deploy and Verify. Keep that difference visible until the stage order and target separation are corrected. A staging-first sequence is shown below as the required design.

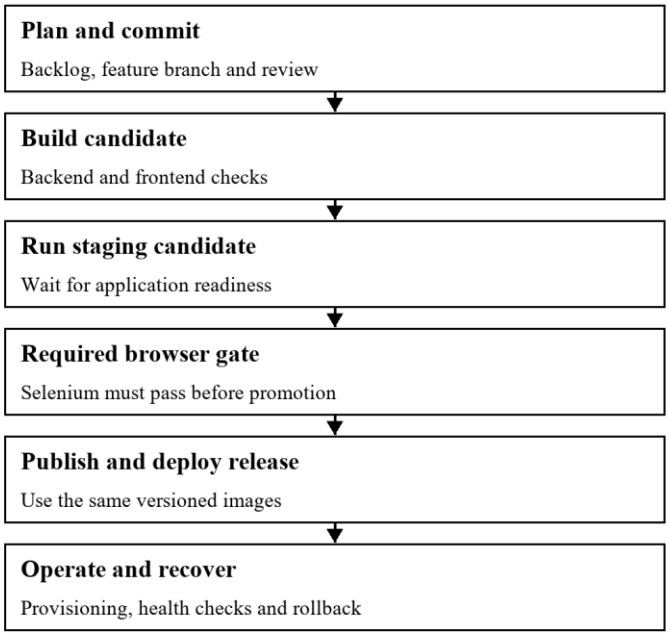


Figure D1 Required promotion order. This diagram is a completion plan, not a screenshot of an executed Jenkins build.

Create an isolated exercise branch. Introduce a harmless defect, such as an incorrect expected dashboard label, and run the staging gate. Capture the failing assertion and skipped release stage. Correct the defect in a real commit and rerun. The failed and passed screenshots must show their actual build numbers and revisions.

Appendix E

Ansible control machine and first provisioning

Use WSL or a Linux control machine and an approved Ubuntu target. Replace the example inventory address, user and key path. These commands configure host packages and services, so select the target deliberately and use --limit. The private key itself is never a screenshot subject.

```
cd /mnt/c/Users/Eshan/Downloads/broker_lead_management/ansible
ansible --version
ansible-inventory --graph
ansible blms_servers --limit app-server-01 -m ping
ansible-playbook playbook.yml --syntax-check
# Supply DB_PASSWORD privately or use encrypted Ansible Vault variables.
ansible-playbook playbook.yml --limit app-server-01 --ask-become-pass
```

State to inspect on the target

Area	Evidence to collect
Operating system	Hostname and Ubuntu release used for the demonstration.
Packages	Installed Docker, Java, curl and other required utilities.
Identity	id blmsuser and the application group.
Directories	/opt/blms/config, deploy and log directories.
Service	systemctl status docker and docker info.
Private configuration	File exists with restrictive permissions; do not print its contents.

The current ansible.cfg disables host-key checking. Before using an untrusted network or presenting a hardened target, enable normal host-key verification and populate known_hosts. Treat the inventory and firewall behaviour as configuration to review, not proof that a host is secure.

Retain the first PLAY RECAP with actual failed and unreachable counts. If a package is unavailable for the selected Ubuntu release, record the error, correct the package setup and rerun before claiming a successful provision.

Appendix F

Idempotency deployment and rollback runbook

Use the same target and variables for the idempotency comparison. Run the provisioning playbook twice and inspect any changed tasks on the second run. A command that reports no change does not prove that no side effect occurred; inspect the host and container state as well.

```
ansible-playbook playbook.yml --limit app-server-01
ansible-playbook playbook.yml --limit app-server-01
ansible-playbook deploy-playbook.yml --limit app-server-01 -e image_tag=CANDIDATE_TAG
ansible-playbook rollback-playbook.yml --limit app-server-01 -e rollback_version=PREVIOUS_TAG
```

Replace CANDIDATE_TAG and PREVIOUS_TAG with real published tags. Before rollback, confirm the earlier images exist and are compatible with the current schema. Create a uniquely named test lead and note its identifier so persistence can be checked after application replacement.

On the target after each operation

```
docker ps
docker inspect blms-backend-container --format "{{.Config.Image}}"
curl -f http://localhost:8081/api/v1/health
curl -f http://localhost/nginx-health
```

The deployment Compose file is stored under /opt/blms/deploy and its private variables under /opt/blms/config/.env. When running Compose manually on that node, use the same --env-file and -f arguments as the playbook. Do not capture the environment file contents.

Rollback recreates the backend and frontend containers and can briefly interrupt service. It does not undo schema changes or recover deleted rows. Capture the actual image reference, successful health response and preserved test lead, then describe only the recovery behaviour that was observed.

Appendix G

Authentication and user API reference

The API is versioned under /api/v1. Except for health, CSRF-token retrieval and the public login/register routes, application data requires a server session. Obtain a CSRF token before a state-changing request and send its returned header name and token with the session cookies.

Method and path	Access	Behaviour
GET /auth/csrf	Public	Returns token and headerName; establishes the CSRF cookie.
POST /auth/login	Public with CSRF	Verifies password and active state, then saves session identity.
POST /auth/register	Public with CSRF	Creates BROKER accounts only; rejects privileged requested roles.
GET /auth/me	Authenticated	Returns the current server-verified user.
POST /auth/logout	Authenticated with CSRF	Invalidates the session and clears the CSRF cookie.
GET /users	ADMIN	Lists user response objects without password hashes.
GET /users/{id}	ADMIN	Returns one user or 404.
POST /users	ADMIN with CSRF	Creates an account with a valid role.
PUT /users/{id}	ADMIN with CSRF	Updates supported identity, role and active fields.
DELETE /users/{id}	ADMIN with CSRF	Deactivates the account rather than deleting its row.

All paths in the table are relative to /api/v1. Account activation and role are rechecked for later requests. Passwords use BCrypt. Public registration and user creation require at least eight characters and at most 72 UTF-8 bytes. No JWT or browser-stored role is used as the backend authentication authority.

Appendix H

Lead and health API reference

Method and path	Access	Behaviour
GET /leads	Authenticated	Lists all leads or filters using the status parameter.
GET /leads/{id}	Authenticated	Returns one lead or 404.
GET /leads/search?q=...	Authenticated	Searches name, email, phone or source.
POST /leads	Authenticated with CSRF	Creates a NEW lead with timestamps.
PUT /leads/{id}	Authenticated with CSRF	Updates supported lead fields; closing status is role-restricted.
PATCH /leads/{id}/status	Authenticated with CSRF	Changes status; CONVERTED/LOST require MANAGER or ADMIN.
DELETE /leads/{id}	MANAGER or ADMIN with CSRF	Deletes an existing lead or returns 404.
GET /dashboard/summary	Authenticated	Returns totalLeads and countsByStatus.
GET /health	Public	Returns UP after SELECT 1, or DOWN with 503 on database failure.

All paths are relative to /api/v1. The implemented lead listing has no page/size pagination, broker parameter or date-range query. The original Week 3 API table described some of these as planned behaviour; they should not be listed as implemented endpoints.

Expected error handling

Missing authentication is rejected, insufficient role or missing CSRF is forbidden, invalid supported input is a bad request, a missing record returns 404 and a conflicting record can return 409. Frontend mutation errors are displayed to the user. They do not create or delete substitute local records.

Manual API proof

For screenshots, use browser developer tools or an API client with a cookie jar. Show the request method, path, response status and safe response fields. Hide passwords, session cookies and CSRF token values. A broker denial should come from the server, not only from a disabled button.

Appendix I

Data model and source reconciliation

Entity	Implemented fields	Important boundary
Lead	id, name, contactPhone, contactEmail, source, notes, status, assignedBrokerId, createdBy, updatedBy, createdAt, updatedAt	Name and status are non-null. Notes have length 2000. Broker/audit IDs are nullable values, not established JPA relationships.
User	id, name, email, password, role, active, createdAt	Email is unique. Password stores a BCrypt hash. Role is an enum. Active is checked during login and later requests.

LeadStatus contains NEW, CONTACTED, QUALIFIED, CONVERTED and LOST. UserRole contains BROKER, MANAGER and ADMIN. There is no separate Role table in the current model. User uses name and email; the earlier proposed username, fullName and passwordHash names are not the actual Java field names.

Earlier wording	Current reference
MySQL in setup prose	PostgreSQL is the application database.
application.yml	backend/src/main/resources/application.properties
Backend port 8080	Backend port 8081
Standalone Tomcat WAR	Executable JAR with embedded Tomcat
Browser localStorage login	Server session with CSRF-protected writes
All requirements complete	Use the requirement-by-requirement gap register.
Zero downtime rollback	Container recreation with a possible brief interruption

These changes reconcile the earlier reports with the current source. They do not retroactively certify that the original setup demonstrations occurred. Keep historical screenshots labelled with their actual environment and capture fresh evidence for the final baseline.

Appendix J

Open requirements and completion work

Item	Current gap	Evidence or completion action
Broker ownership	All authenticated users can list leads.	Implement assignment and broker scoping, then test cross-user access.
Actor audit	createdBy and updatedBy are not filled by LeadService.	Populate from server identity and verify database values.
Strict transitions	Roles restrict closing, but no strict next-state graph is enforced.	Agree the transition rules, implement them and add negative tests.
Original required fields	Contact/source requirements differ from the current name-only mandatory rule.	Confirm the agreed scope and align validation and tests.
Date filtering	No date-range filter is implemented.	Implement it or document an approved scope reduction.
Failure screenshots	Selenium base class does not capture failures automatically.	Implement and exercise the mechanism in Appendix C.
Browser promotion gate	Browser Tests currently follows deployment.	Use the staging-first order in Appendix D.
Live delivery proof	Jenkins, registry, Docker and Ansible were not executed in this validation.	Capture the real runs requested in Weeks 7 to 15.
Performance targets	No formal timing measurements were recorded.	Measure entry time and dashboard refresh under a stated test setup.
Final revision	Recent fixes were uncommitted during validation.	Review, commit and identify the exact submitted revision.

Other operational work includes schema migrations, database backups, HTTPS setup and pagination for larger datasets. These are limitations or future improvements, not evidence of completed requirements. Prioritise explicit assignment gaps before optional enhancements.

Appendix K

Screenshot register for Weeks 1 to 8

ID	Evidence	File and completion
W01-S01	Problem statement and scope	W01-S01.png Pending
W01-S02	Stakeholders and objectives	W01-S02.png Pending
W01-S03	Success criteria	W01-S03.png Pending
W01-S04	Scope review record	W01-S04.png Pending
W02-S01	Product backlog	W02-S01.png Pending
W02-S02	Kanban task board	W02-S02.png Pending
W02-S03	Fifteen week plan	W02-S03.png Pending
W02-S04	Definition of Done and workflow	W02-S04.png Pending
W03-S01	Tools and versions	W03-S01.png Pending
W03-S02	Backend and database health	W03-S02.png Pending
W03-S03	Working local frontend	W03-S03.png Pending
W03-S04	Architecture and data model	W03-S04.png Pending
W04-S01	Repository home	W04-S01.png Pending
W04-S02	Initial commit history	W04-S02.png Pending
W04-S03	Issues and templates	W04-S03.png Pending
W04-S04	Branch policy and protection	W04-S04.png Pending
W05-S01	Feature branch and commits	W05-S01.png Pending
W05-S02	Working create workflow	W05-S02.png Pending
W05-S03	Pull request review	W05-S03.png Pending
W05-S04	Merge evidence	W05-S04.png Pending
W06-S01	Complete lead workflow	W06-S01.png Pending
W06-S02	Second feature branch	W06-S02.png Pending
W06-S03	Conflict and resolution	W06-S03.png Pending
W06-S04	Release tag and backlog	W06-S04.png Pending
W07-S01	Jenkins installation and tools	W07-S01.png Pending
W07-S02	Job and repository settings	W07-S02.png Pending
W07-S03	Automatic build cause	W07-S03.png Pending
W07-S04	Successful build and archive	W07-S04.png Pending
W08-S01	Jenkinsfile stages	W08-S01.png Pending
W08-S02	Pipeline execution graph	W08-S02.png Pending

Use the exact ID as the filename, insert it in the matching weekly slot and change Pending only after reviewing the image. The capture procedure is beside the slot. Week 8 continues across the two register pages.

Appendix L

Screenshot register for Weeks 8 to 15

ID	Evidence	File and completion
W08-S03	Application through Nginx	W08-S03.png Pending
W08-S04	Parameterised configuration	W08-S04.png Pending
W09-S01	Selenium code and assertions	W09-S01.png Pending
W09-S02	Local Maven suite result	W09-S02.png Pending
W09-S03	Surefire report	W09-S03.png Pending
W09-S04	Automatic failure screenshot	W09-S04.png Pending
W10-S01	Published Jenkins tests	W10-S01.png Pending
W10-S02	Failed test blocks deployment	W10-S02.png Pending
W10-S03	Defect correction commit	W10-S03.png Pending
W10-S04	Successful rerun	W10-S04.png Pending
W11-S01	Dockerfiles and build	W11-S01.png Pending
W11-S02	Image details	W11-S02.png Pending
W11-S03	Running stack and logs	W11-S03.png Pending
W11-S04	Container lifecycle	W11-S04.png Pending
W12-S01	Versioned Jenkins image build	W12-S01.png Pending
W12-S02	Registry publication	W12-S02.png Pending
W12-S03	Running selected image	W12-S03.png Pending
W12-S04	Commit to container run	W12-S04.png Pending
W13-S01	Prerequisites and inventory	W13-S01.png Pending
W13-S02	Control machine connectivity	W13-S02.png Pending
W13-S03	First provisioning run	W13-S03.png Pending
W13-S04	Provisioned host state	W13-S04.png Pending
W14-S01	Clean target and first run	W14-S01.png Pending
W14-S02	Idempotent rerun	W14-S02.png Pending
W14-S03	Deployment and health	W14-S03.png Pending
W14-S04	Rollback and recovery	W14-S04.png Pending
W15-S01	Final repository baseline	W15-S01.png Pending
W15-S02	End to end pipeline	W15-S02.png Pending
W15-S03	Live final application	W15-S03.png Pending
W15-S04	Provisioning and final demonstration	W15-S04.png Pending

Use the exact ID as the filename, insert it in the matching weekly slot and change Pending only after reviewing the image. The capture procedure is beside the slot. Week 8 continues across the two register pages.

Appendix M

Troubleshooting and viva preparation

Symptom	Check and response
Frontend opens but API fails	Check the backend process and port 8081. Inspect the Vite proxy and health response.
Database login failure	Match the configured database and existing credentials. A changed environment value does not reset an existing PostgreSQL volume.
Login works but a write returns 403	Confirm the session cookie, CSRF request and user role. Do not disable protection as a workaround.
Selenium cannot reach the app	Check BASE_URL and readiness. Start the complete app; an unavailable target should fail the suite.
Chrome session fails to start	Check the installed browser, compatible driver and agent permissions. Retain the actual error.
Docker engine unavailable	Start the Linux engine and check docker version before interpreting Compose output.
Ansible unreachable or sudo denied	Check inventory, SSH user, key and become permissions on the selected target.
Rollback uses the wrong version	Check the published tag, Compose interpolation and the running container image reference.

Questions to practise

- Why are role checks required on the backend even when the UI hides an action?
- What does a server session do, and why does a cookie-authenticated write need CSRF protection?
- How does the reload assertion improve a lead-creation test?
- What is the difference between an image, a container and a persistent volume?
- Why must the Selenium gate run before promotion to the release target?
- How would you prove idempotency, and what does image rollback leave unchanged?

Answer using the actual source and demonstration. Avoid claiming a database guarantee, review event or deployment result that you cannot show.

Appendix N

Sources and final submission checklist

Project sources

1. Selenium Testing for a Broker Lead Management System.pdf. Supplied two-page assignment brief containing fifteen tasks.

2. BLMS_Week1_Problem_Definition_and_Scope_1.docx. Supplied Week 1 problem, objectives, stakeholders and scope.

3. BLMS_Week2_Agile_Planning_and_DevOps_Workflow.pdf. Supplied four-page planning report. Referenced companion workbook was not supplied.

4. BLMS_Week3_Requirements_Architecture_and_Technology_Setup.pdf. Supplied five-page requirements and architecture report.

5. Current local checkout of https://github.com/esh22nika/broker_lead_management. Controllers, services, models, frontend components and build configuration inspected on 28 October 2026.

6. Jenkinsfile, docker-compose.yml, Dockerfiles and ansible playbooks in the current checkout. Definitions inspected; no live Jenkins or provisioning success inferred.

7. backend/target/surefire-reports. Local backend and Selenium reports with zero failures and skips. Frontend npm test and build command outputs recorded in the work session.

8. README.md, CONTRIBUTING.md, CHANGELOG.md and docs/BACKLOG.md. Historical milestone documents were treated as descriptions to reconcile, not independent execution proof.

Before submission

- Replace the required screenshot spaces with actual images and complete their captions.
- Resolve the explicit assignment gaps or obtain and record an agreed scope decision.
- Use one final source revision, Jenkins build and release tag consistently.
- Hide secrets while keeping evidence identifiers readable.
- Update the Word contents field and page fields after inserting screenshots.
- Check every table, page break and screenshot at readable zoom.
- Add the actual submission date and signatures only after final review.
- Prepare the separate presentation and live demonstration requested by the brief.

Appendix O

Business stories and acceptance conditions

These eight stories preserve the business meaning of the supplied Week 2 report. Their original identifiers are retained here. The later repository backlog uses a different ordering, so match stories by meaning when tracing them to implementation.

Original story	Acceptance conditions and current boundary
US 01 Create a lead	A broker submits the required details, receives validation for missing mandatory input and sees the saved NEW lead. Current validation requires a name; agree the original contact/source rule.
US 02 View assigned leads	A broker sees the leads assigned to that broker and a clear empty state when none exist. Listing exists, but assignment scoping remains unfinished.
US 03 Update details and status	An edit persists, the status reflects the change and updatedAt changes. Verify with a reload and the backend response.
US 04 Search and filter	A keyword narrows results and a selected status filters the pipeline. Keyword and status filtering exist; the original date-filter condition remains incomplete.
US 05 Control status workflow	Brokers can select only permitted next states and managers can override according to agreed rules. Role-based closing restrictions exist, but a strict transition graph does not.
US 06 Review the summary	Counts per status are accurate and refresh after a status change. Test counts using known records and separately measure the five-second freshness target.
US 07 Administer access	A broker is denied administrator operations, while an administrator can create, edit and deactivate users. Verify denial at the API as well as the visible interface.
US 08 Sign in securely	Correct credentials open the application, incorrect credentials are rejected and logout removes authenticated access. Server-session and browser checks cover these paths.

The acceptance conditions are a review baseline. Any reduced requirement should be agreed and recorded; a changed implementation should not silently redefine the original story.

Appendix P

Stakeholders scope and Definition of Done

Stakeholder	Interest and review role
Broker	Captures enquiries and maintains lead details during daily work.
Manager	Reviews team pipeline and controls closing decisions.
Administrator	Maintains accounts, roles and the operating environment.
Developer or project owner	Maintains the backlog, code, tests and delivery configuration.
Data owner	Reviews data integrity and the proposed audit trail.
Evaluator	Checks the weekly deliverables, evidence and final demonstration.

Definition of Done from the planning baseline

- An accepted story has clear acceptance conditions and its behaviour has been checked.
- Code has been reviewed through the agreed pull request process and comments have been addressed.
- Relevant unit, integration and browser tests pass for the submitted revision.
- The change has no known blocker and has no unresolved merge conflict.
- Documentation reflects changes to setup, the API or data model.
- The required pipeline and deployment evidence is available before release sign-off.

Scope remains limited to a web lead-management MVP and its DevOps process. Notifications, external integrations, advanced reporting, native mobile development and multi-branch operation remain excluded. CSV export is an implemented enhancement; it does not replace a missing core acceptance condition.

Appendix Q

Nonfunctional requirements and measurement plan

Requirement	How to measure or inspect	Evidence state
Lead creation under two seconds server-side	Record API timing under a stated environment and dataset. Repeat enough requests to describe a meaningful sample.	Target only; not benchmarked in this report.
Dashboard freshness within five seconds	Time from a committed status update to refreshed correct counts. State whether manual refresh is used.	Functional counts checked; timing still needed.
Lead entry under two minutes	Time a user entering a representative synthetic enquiry, including validation corrections.	Usability measurement pending.
Password and role security	Inspect BCrypt storage and test anonymous, broker, manager and administrator requests.	Local security integration tests passed.
Persistence and reliability	Reload after writes, recreate application containers and inspect stored records. Use backups for recovery claims.	Browser reload checked; container and backup proof pending.
Portable deployment	Build the executable JAR and frontend assets, then start the selected server/container environment.	Local builds passed; target deployment proof pending.
Versioned maintainable API	Inspect /api/v1 routes and trace behaviour to controllers, service and repository.	Implemented; actual endpoint list is in Appendices G and H.
Testable core journeys	Use backend and browser assertions tied to the agreed acceptance conditions.	Passing tests cover exercised paths, not every possible input.

The use cases follow the three application roles: a broker records and maintains leads, a manager reviews the pipeline and closes outcomes, and an administrator manages users. Ownership isolation and strict transition rules remain separate acceptance gaps. Avoid describing an unmeasured target as a result or a finite test sample as a guarantee.

Appendix R

Use case diagram and role boundaries

The three actors are Broker, Manager and Administrator. Common lead operations and the summary dashboard are available to authenticated users. Closing or deleting a lead requires a manager or administrator, and account administration requires an administrator. The diagram describes the current backend permissions; it does not imply that leads are already isolated by assigned broker.

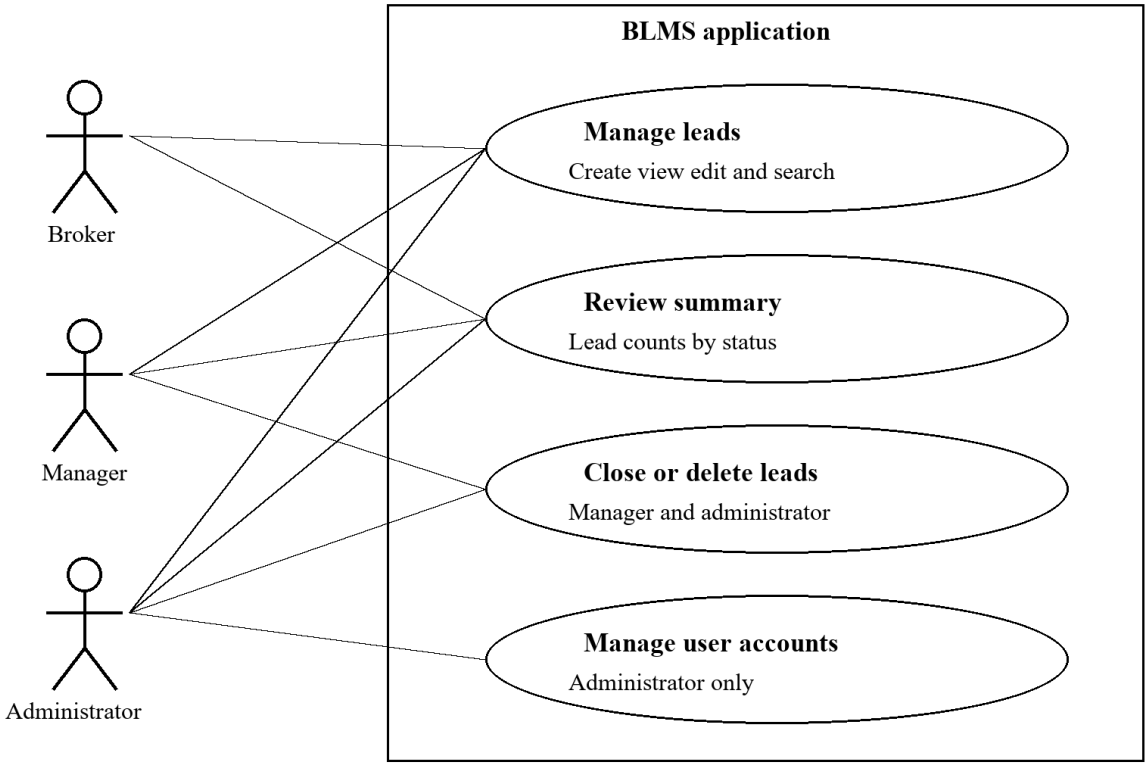


Figure R1 Use cases grouped by current server-side permission. Detailed acceptance conditions are retained in Appendix O.

The original requirements additionally call for broker ownership and strict next-state transitions. Those are separate implementation tasks in Appendix J. A broker denial screenshot should show the protected request returning a server error status, while a manager or administrator can complete the corresponding permitted action.